

Machine Learning for Biology and Health

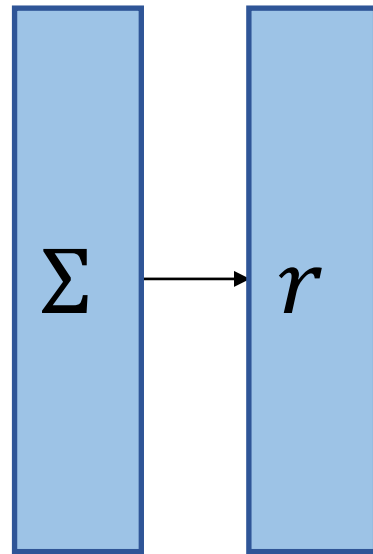
CSCI 1851
Spring 2026

Ritambhara Singh

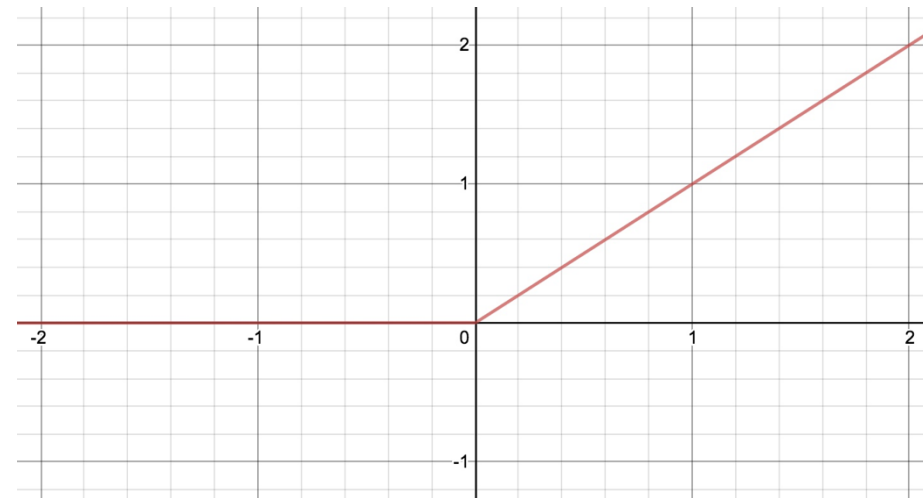
March 17, 2026
Tuesday



ReLU



Rectified Linear Unit



$$\text{ReLU}(x) = \max(0, x)$$

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

Today's goal – Learn about CNNs and regularization for deep learning

- (1) Convolution in Pytorch
- (2) Building our CNN model
- (3) Maxpooling
- (4) Regularization for deep learning models
- (5) Class activity: CNN and regularization demo

Convolution in PyTorch

Convolution in PyTorch

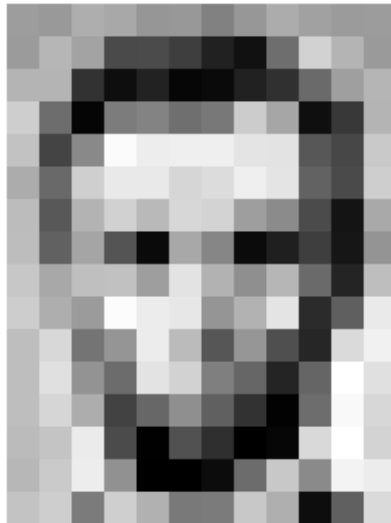
`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Can you guess the shape of the X-ray image?

Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

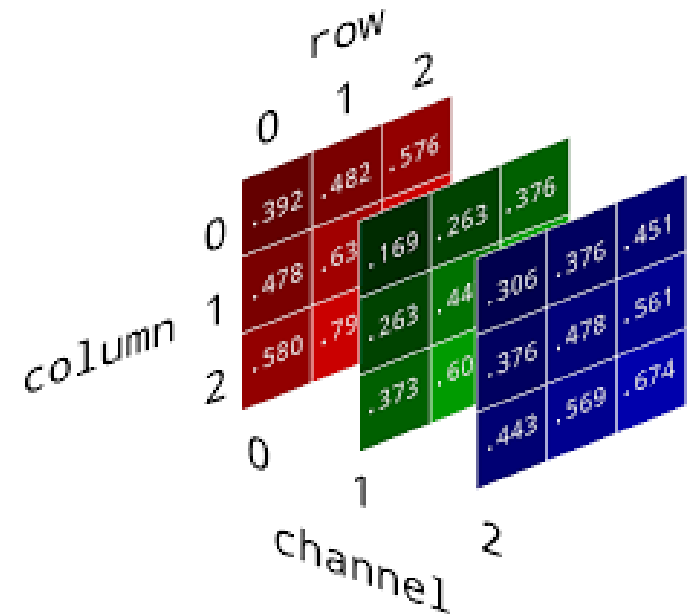
Convolution in PyTorch

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Convolution in PyTorch

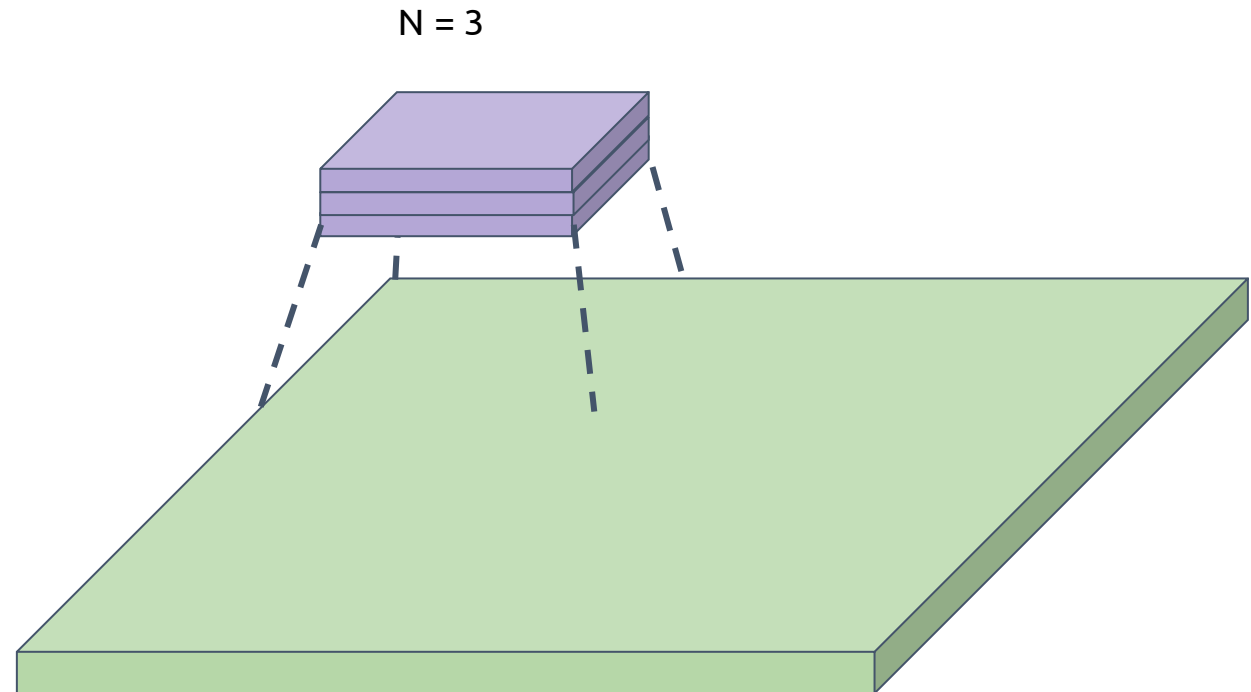
`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Output Size of a Convolution Layer

The output size of a convolution layer depends on 4 Hyperparameters:

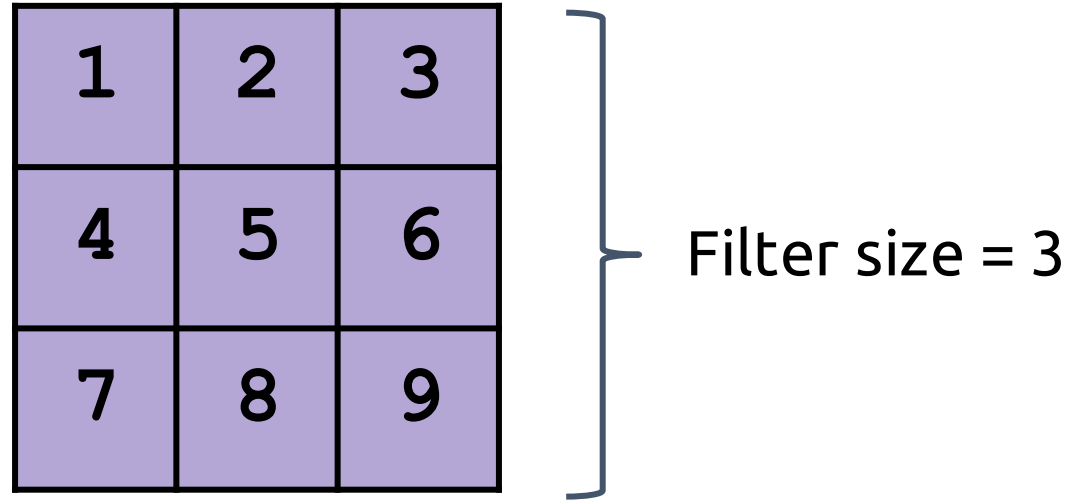
- Number of filters, **N**



Output Size of a Convolution Layer

The output size of a convolution layer depends on 4 Hyperparameters:

- Number of filters, **N**
- The size of these filters, **F**



Convolution in PyTorch

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Output Size of a Convolution Layer

The output size of a convolution layer depends on 4 Hyperparameters:

- Number of filters, **N**
- The size of these filters, **F**
- The stride, **S**

2	0	3	1	0
2	4	5	2	3
0	0	3	3	1
2	9	9	7	8
3	4	7	2	1

Stride = 2



2	0	3	1	0
2	4	5	2	3
0	0	3	3	1
2	9	9	7	8
3	4	7	2	1

Convolution in PyTorch

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Convolution in PyTorch

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

“Problem” With Convolution

2	0	1	3
0	1	1	0
0	0	2	0
0	1	1	1

 $\otimes$

1	1	1
0	0	0
-1	-1	-1

 $=$

1	2
0	-1

- Output of convolution is always smaller than the input
- Why might we want the output size to be the same?
 - To avoid the filter “eating at the border” of the image when applying multiple conv layers

Solution: Padding

Apply the kernel to 'imaginary' pixels surrounding the image

2	0	3	1	1
1	1	0	0	2
4	3	2	0	1
1	0	5	2	0
0	1	0	3	0

Solution: Padding

Apply the kernel to 'imaginary' pixels surrounding the image

?	?	?	?	?	?	?
?	2	0	3	1	1	?
?	1	1	0	0	2	?
?	4	3	2	0	1	?
?	1	0	5	2	0	?
?	0	1	0	3	0	?
?	?	?	?	?	?	?

What Values to Use For These Pixels?

?	?	?	?	?	?	?
?	2	0	3	1	1	?
?	1	1	0	0	2	?
?	4	3	2	0	1	?
?	1	0	5	2	0	?
?	0	1	0	3	0	?
?	?	?	?	?	?	?

What Values to Use For These Pixels?

Standard practice: fill with zeroes

0	0	0	0	0	0	0
0	2	0	3	1	1	0
0	1	1	0	0	2	0
0	4	3	2	0	1	0
0	1	0	5	2	0	0
0	0	1	0	3	0	0
0	0	0	0	0	0	0

Padding Modes in Pytorch

Available options: 'VALID' or 'SAME' or int/tuple

Valid

Filter only slides over
"Valid" regions of the
data

2	0	1	3
0	1	1	0
0	0	2	0
0	1	1	1

Same

Filter slides over the bounds of the
data, ensuring output size is the
"Same" as input size (when stride = 1)

0	0	0	0	0	0
0	2	0	1	3	0
0	1	1	2	3	0
0	4	3	2	1	0
0	8	3	1	3	0
0	0	0	0	0	0

Output Size of a Convolution Layer

The output size of a convolution layer depends on 4 Hyperparameters:

- Number of filters, **N**
- The size of these filters, **F**
- The stride, **S**
- The amount of padding, **P**

0	0	0	0	0	0
0	0	0	0	0	0
0	0	2	3	0	0
0	0	9	2	0	0
0	0	0	0	0	0
0	0	0	0	0	0

} Padding = 2

Output Size of a Convolution Layer

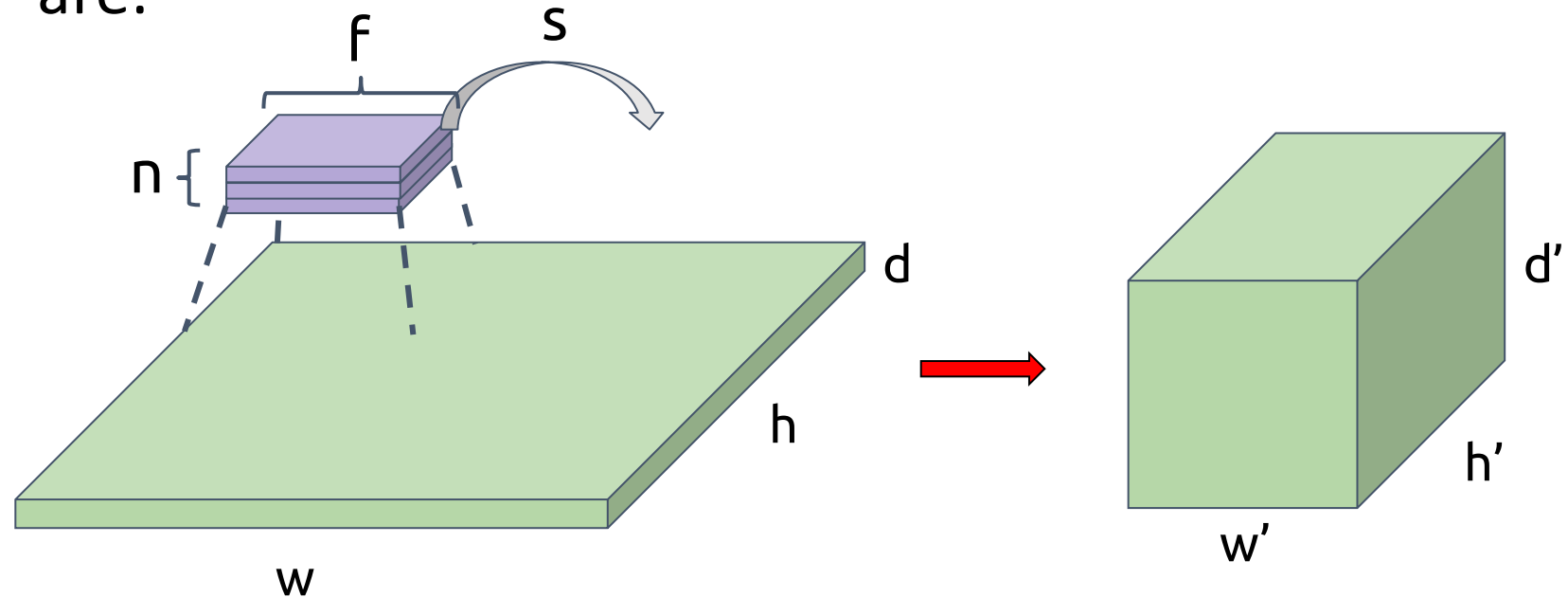
Suppose we know the number of filters, their size, the stride, and padding (**n,f,s,p**).

Then for a convolution layer with input dimension **w x h x d**, the output dimensions **w' x h' x d'** are:

$$w' = \frac{w - f + 2p}{s} + 1$$

$$h' = \frac{h - f + 2p}{s} + 1$$

$$d' = n$$



Output Size for “VALID” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 0$

Let $w = 4$

$$\begin{aligned} w' &= \frac{4 - 3 + 2 \cdot 0}{1} + 1 \\ &= 1 + 1 = 2 \end{aligned}$$

Output Size for “VALID” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 0$

2	0	1	3
0	1	1	0
0	0	2	0
0	1	1	1

$w = 4$

1	



$w' = 2$

Output Size for “VALID” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 0$

2	0	1	3
0	1	1	0
0	0	2	0
0	1	1	1

w = 4

1	2

w' = 2

Output Size for “VALID” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 0$

2	0	1	3
0	1	1	0
0	0	2	0
0	1	1	1

$w = 4$

1	2
3	

$w' = 2$

Output Size for “VALID” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 0$

2	0	1	3
0	1	1	0
0	0	2	0
0	1	1	1

w = 4

1	2
3	4

w' = 2

Output Size for “SAME” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 1^*$

Let $w = 4$

$$\begin{aligned} w' &= \frac{4 - 3 + 2 \cdot 1}{1} + 1 \\ &= 3 + 1 = 4 \end{aligned}$$

*Chosen so output size is the same

Output Size for “SAME” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 1^*$

*Chosen so output size is the same

0	0	0	0	0	0
0	2	0	1	3	0
0	1	1	2	3	0
0	4	3	2	1	0
0	8	3	1	3	0
0	0	0	0	0	0



$w = 4$

1			



$w' = 4$

Output Size for “SAME” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 1^*$

*Chosen so output size is the same

0	0	0	0	0	0
0	2	0	1	3	0
0	1	1	2	3	0
0	4	3	2	1	0
0	8	3	1	3	0
0	0	0	0	0	0



w = 4

1	2		



w' = 4

Output Size for “SAME” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 1^*$

*Chosen so output size is the same

0	0	0	0	0	0
0	2	0	1	3	0
0	1	1	2	3	0
0	4	3	2	1	0
0	8	3	1	3	0
0	0	0	0	0	0



$w = 4$

1	2	3	



$w' = 4$

Any questions?



Output Size for “SAME” Padding

$$w' = \frac{w - f + 2p}{s} + 1$$

num filters $n = 1$

filter size $f = 3$

stride $s = 1$

padding $p = 1^*$

*Chosen so output size is the same

0	0	0	0	0	0
0	2	0	1	3	0
0	1	1	2	3	0
0	4	3	2	1	0
0	8	3	1	3	0
0	0	0	0	0	0



$w = 4$

1	2	3	4



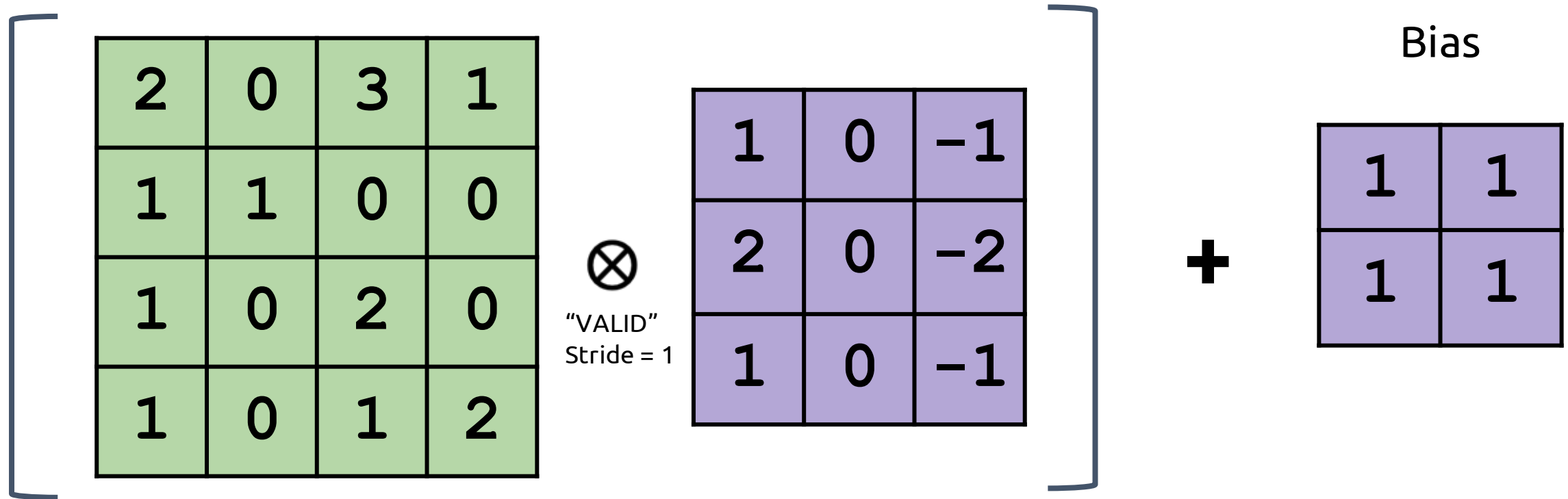
$w' = 4$

Convolution in PyTorch

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

Full documentation here: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Bias Term in Convolution Layers



Just like a fully connected layer, we can have a learnable additive bias for convolution.

Adding a Bias in Pytorch

```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,  
padding=0, bias=True)
```

Building our CNN model

Our problem:

Input: $\mathbb{X}$

Pixel Grid

$x^{(1)} =$



128x128 pixels



Function: f



$f(\mathbb{X}) \rightarrow \mathbb{Y}$

Target: $\mathbb{Y}$

Has Pneumonia?

$y^{(1)} = 0$



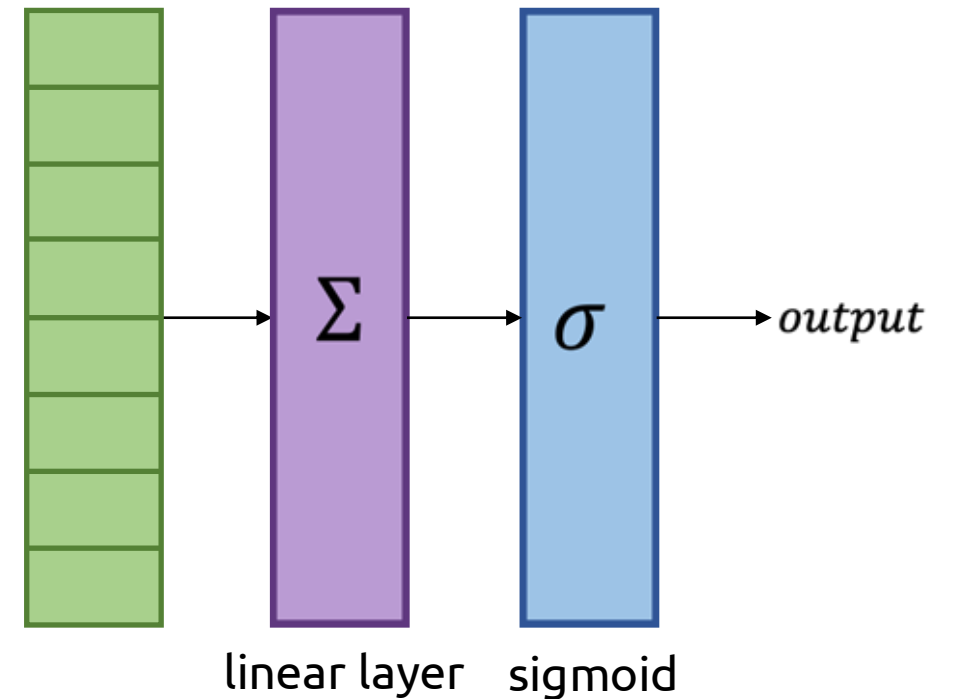
$x^{(2)} =$



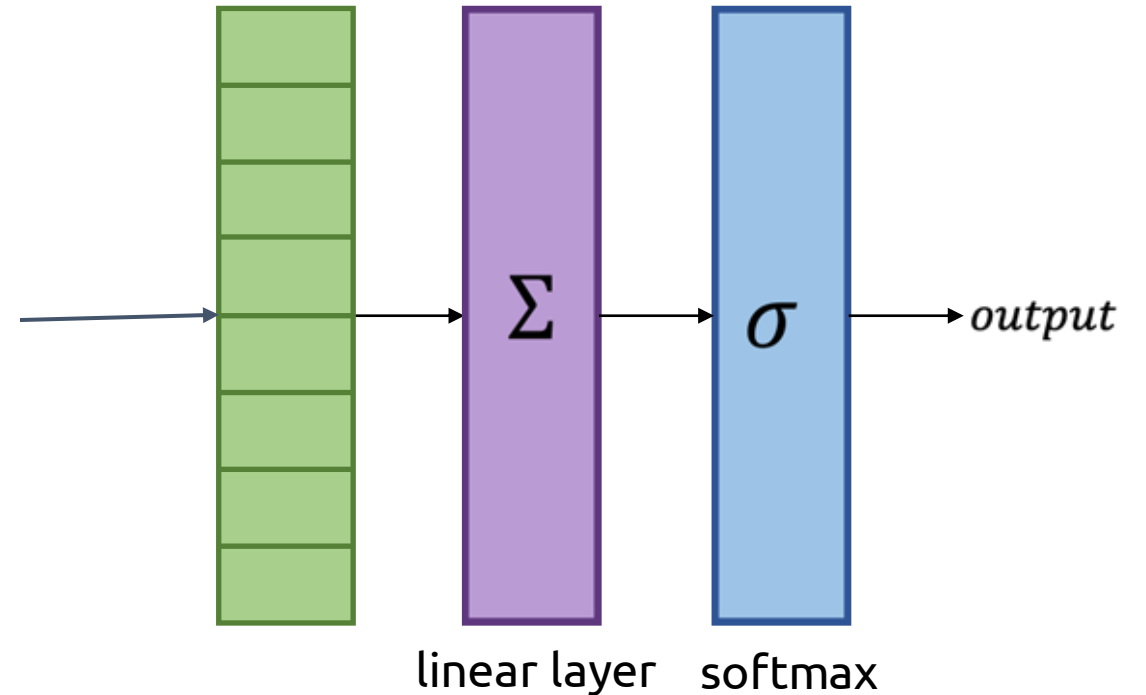
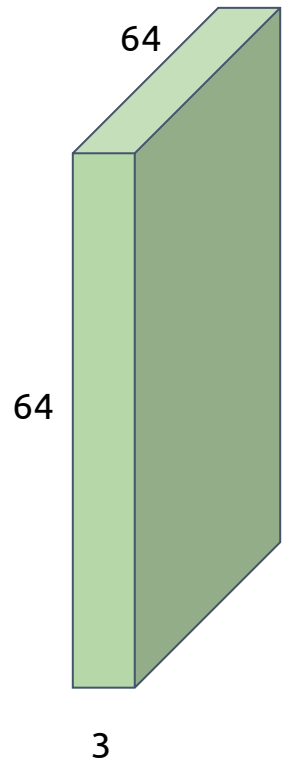
$y^{(2)} = 1$



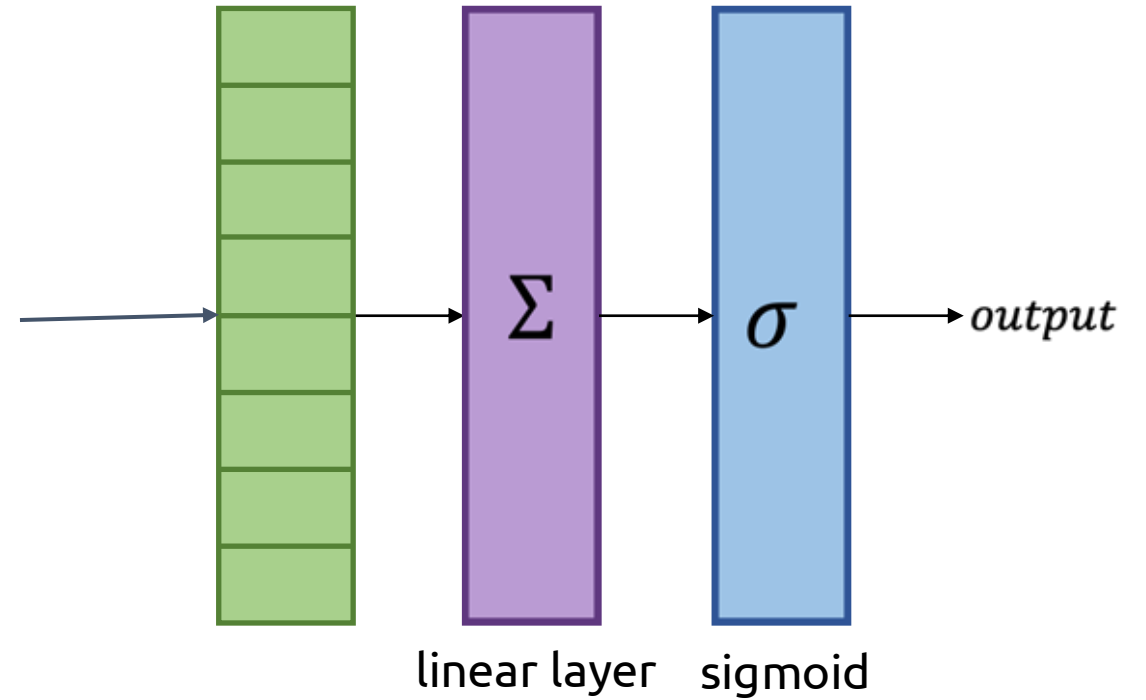
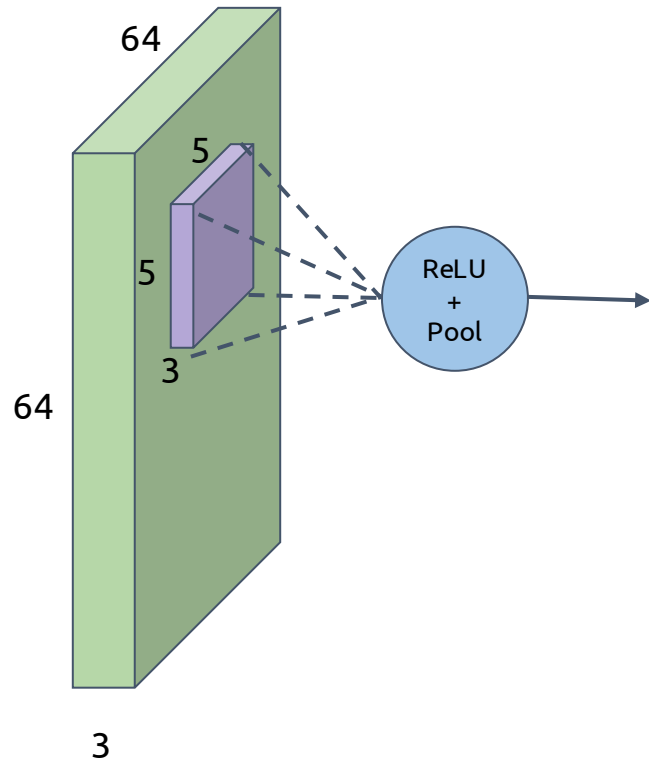
Our neural network so far



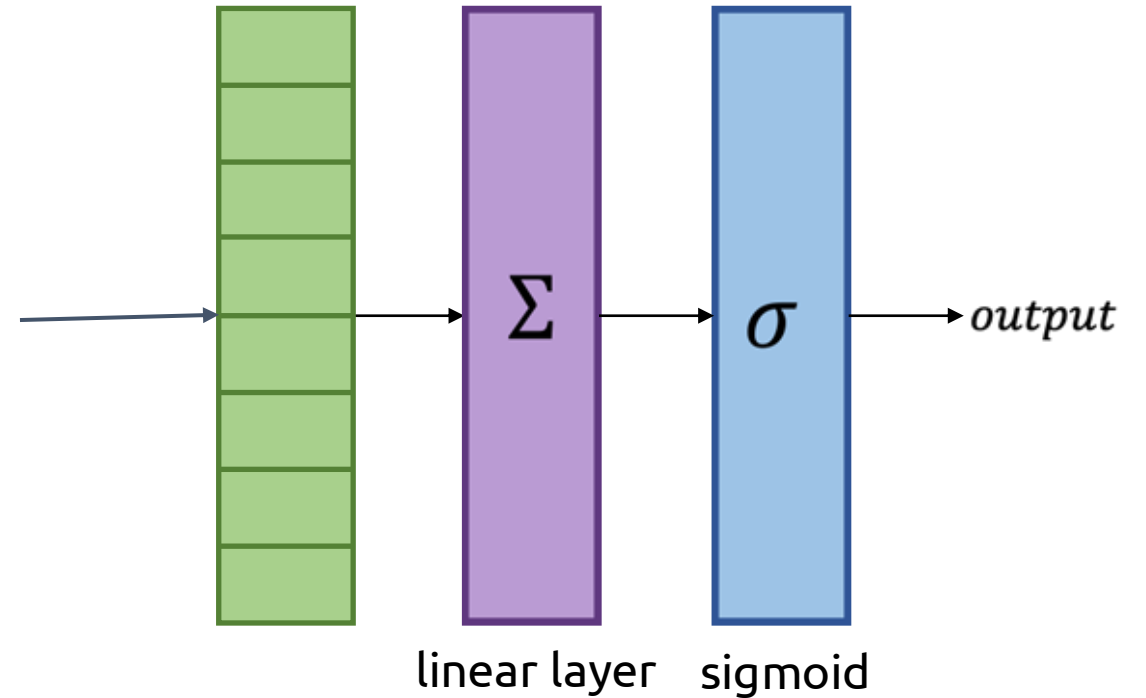
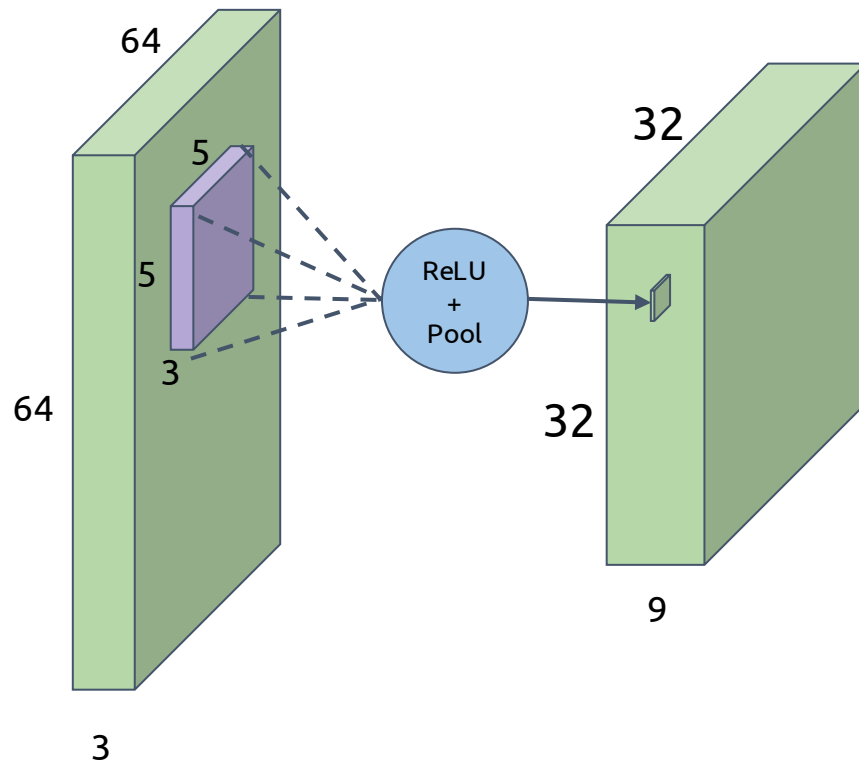
Convolutional Neural Network Architecture



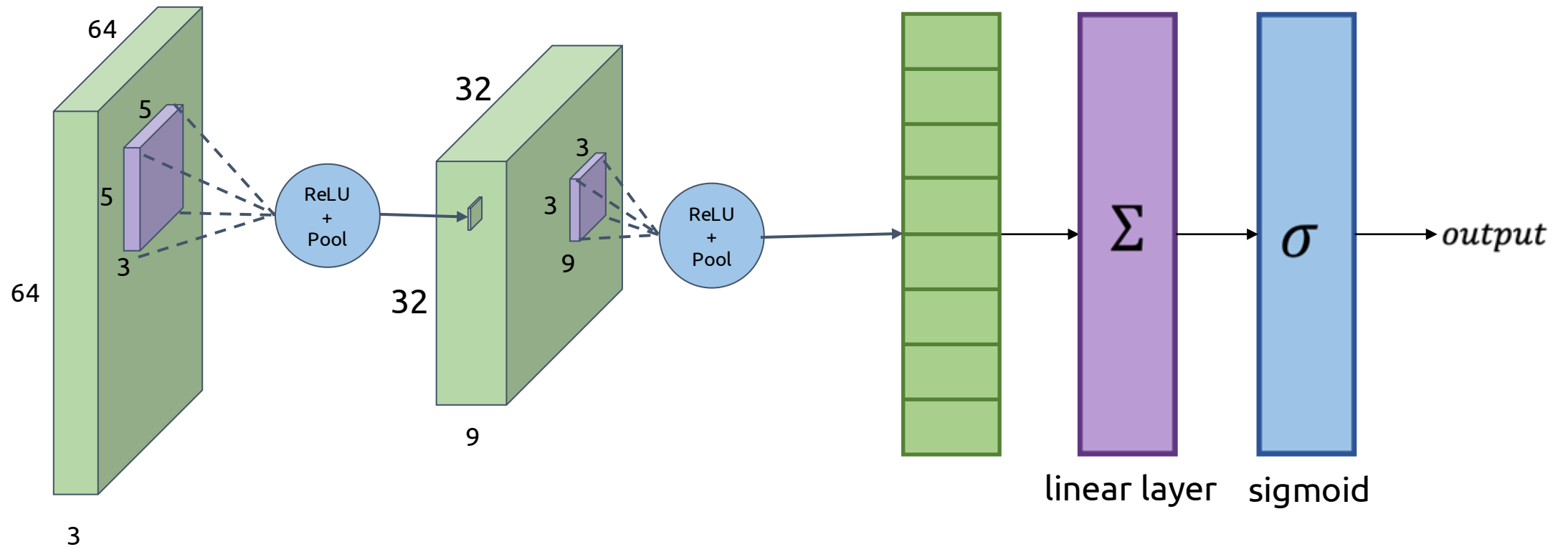
CNN Architecture



CNN Architecture

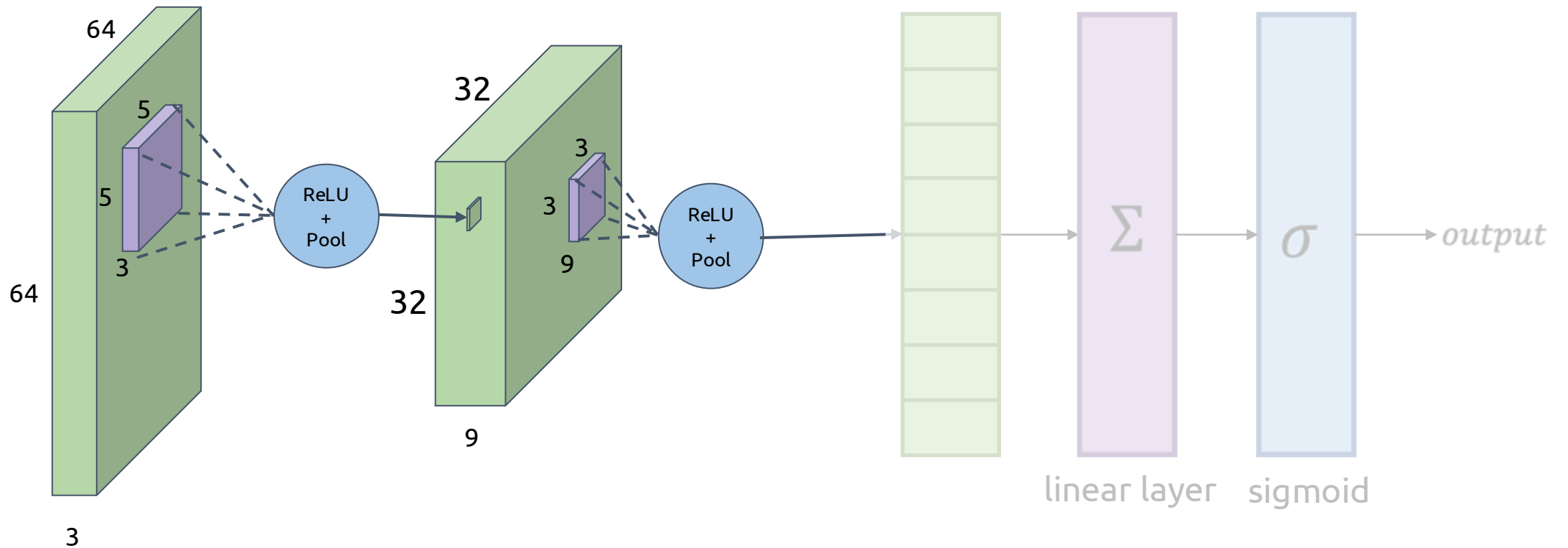


CNN Architecture

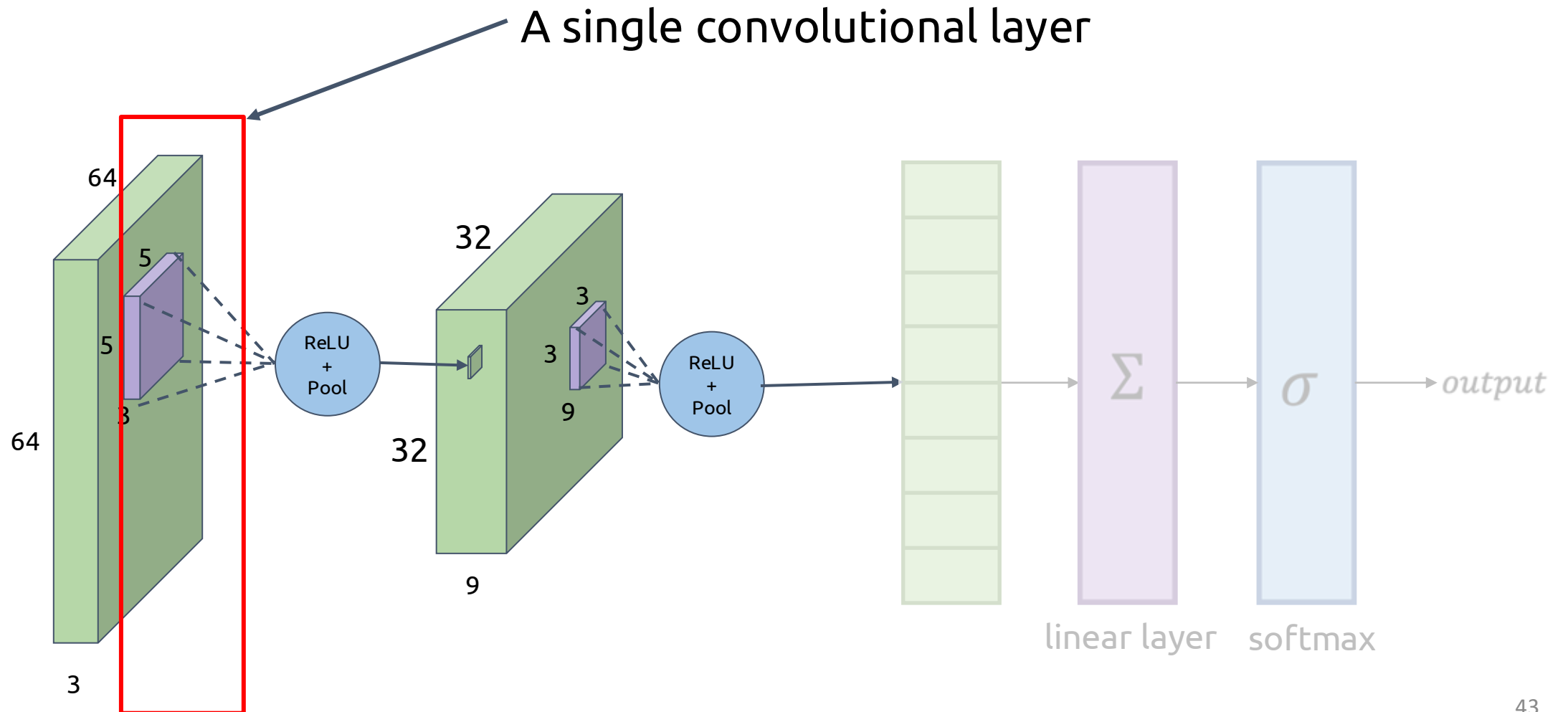


CNN Architecture

This part learns to extract **features** from the image

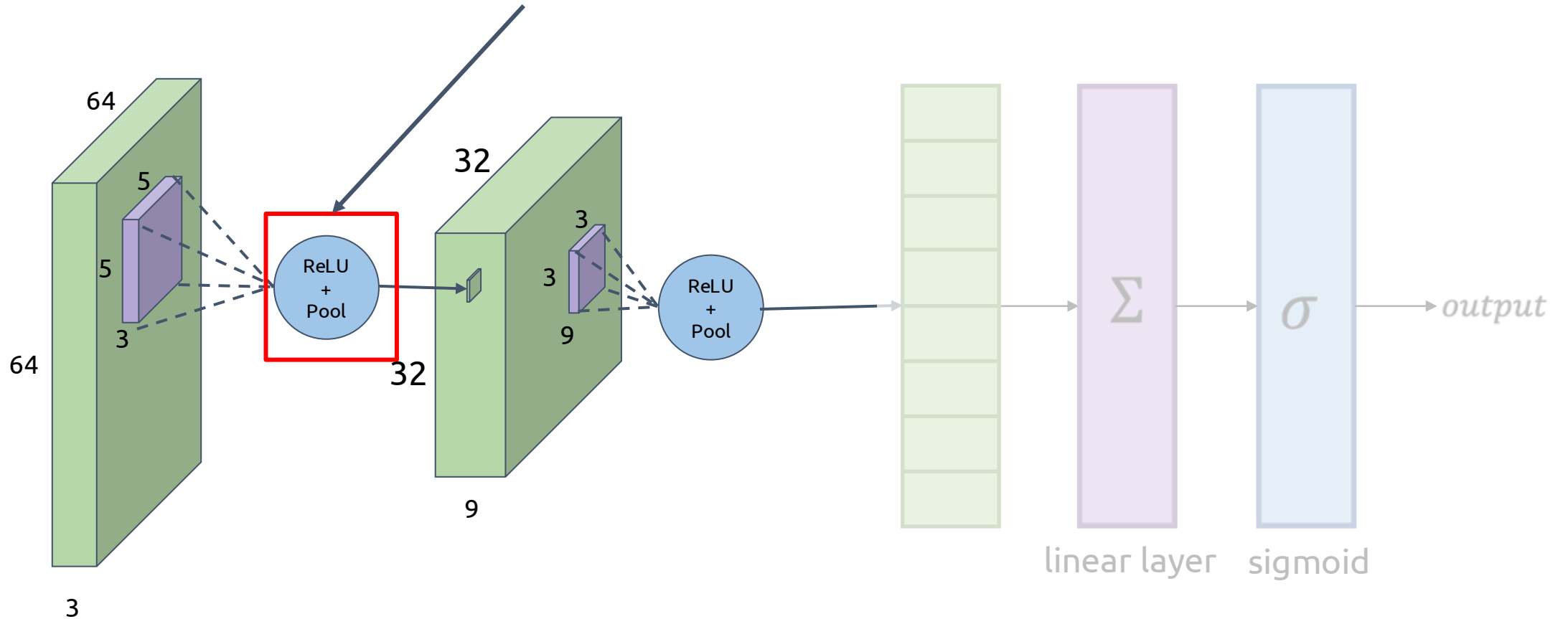


CNN Architecture



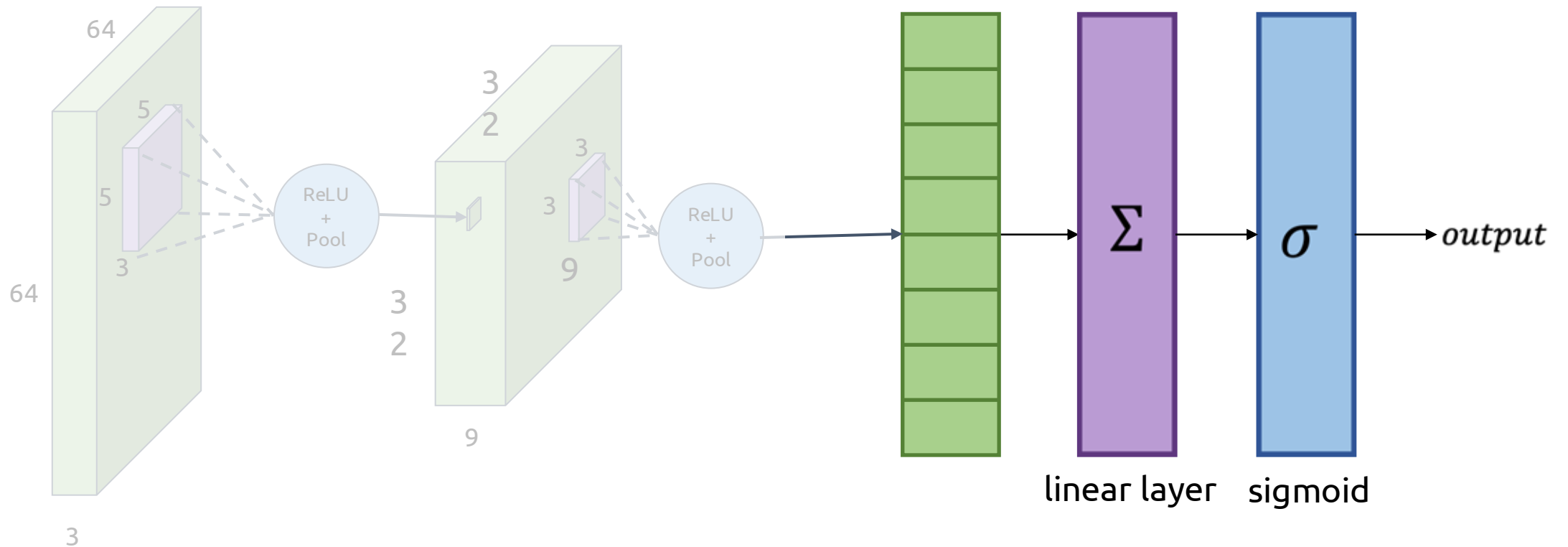
CNN Architecture

Activation after filter passes over image

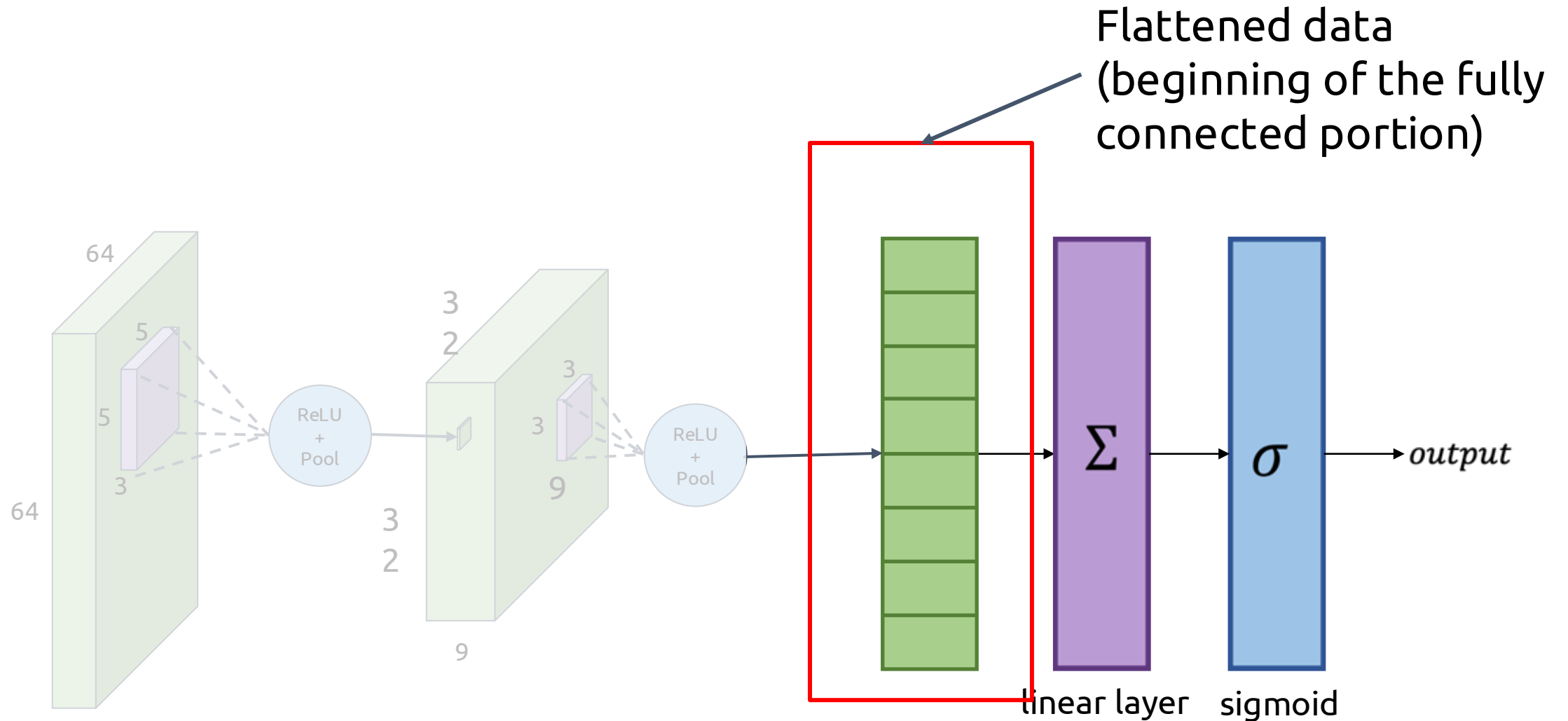


CNN Architecture

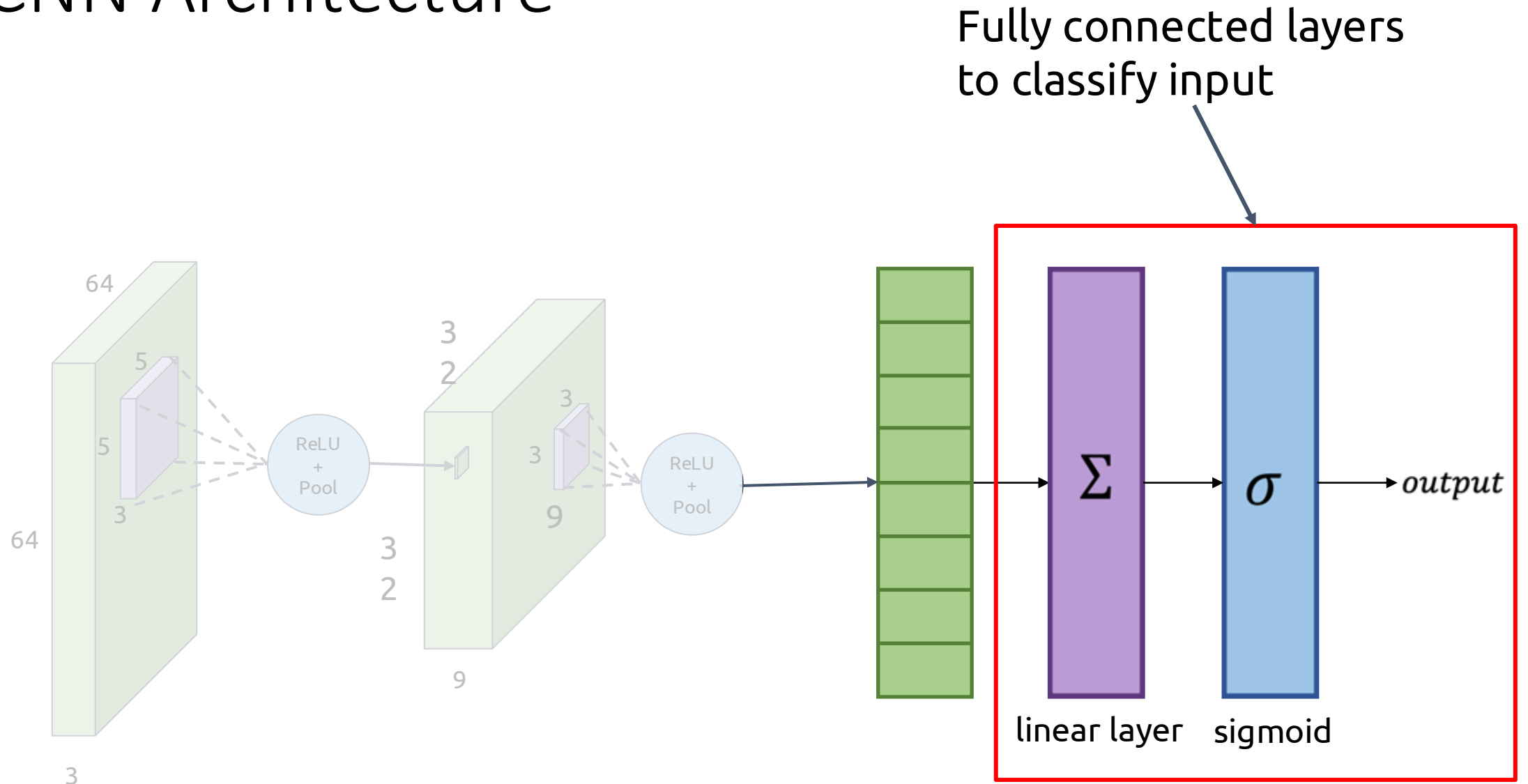
This part learns to perform a specific task
(e.g. classification) using those features



CNN Architecture

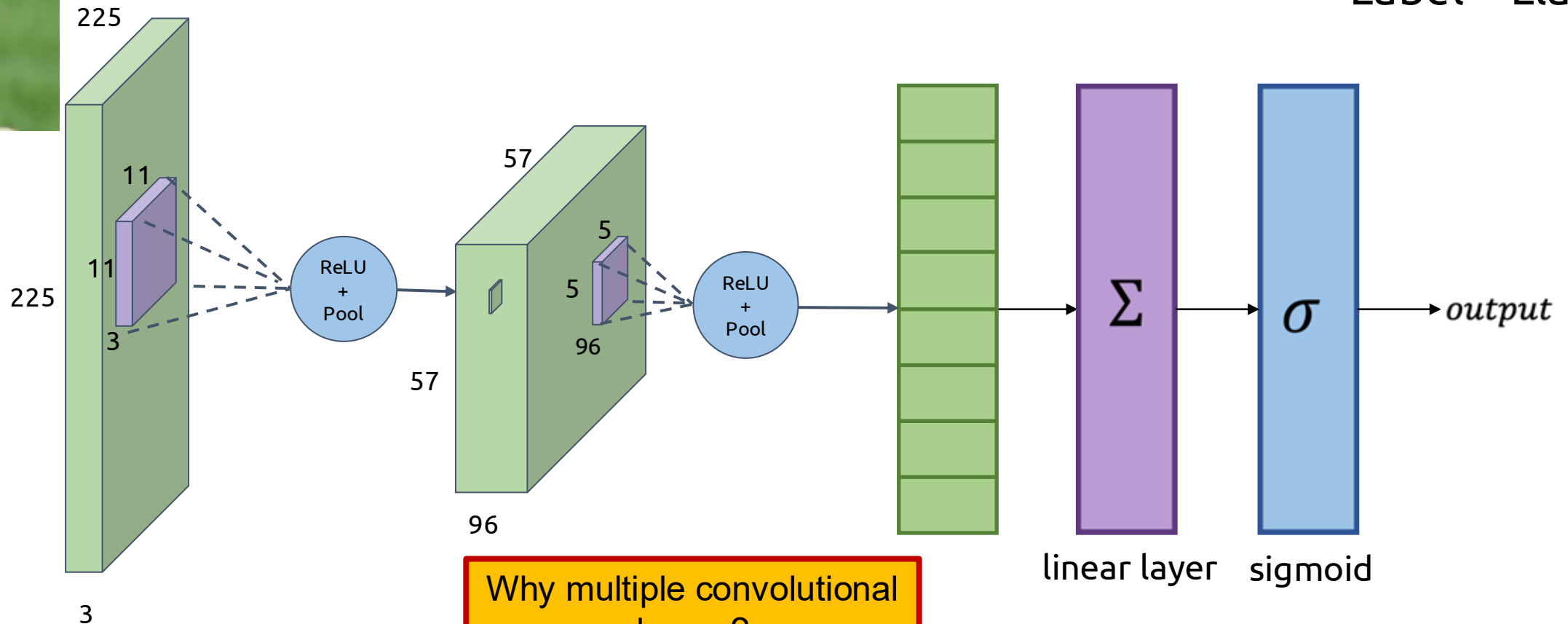


CNN Architecture



CNN Architecture

Input



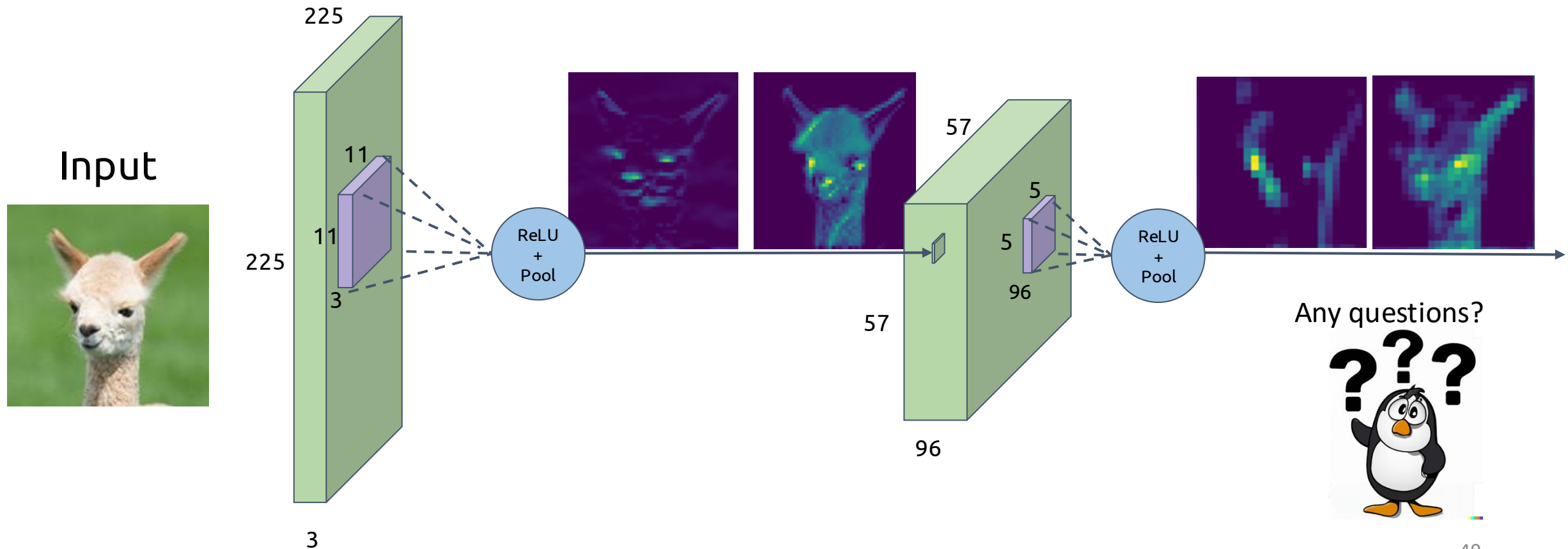
Label="Llama"

Why multiple convolutional layers?

Feature Extraction using multiple convolution layers

Hierarchy of features

Sequence of layers detect broader and broader features



Maxpooling

ILSVRC 2012

([ImageNet](#) Large Scale Visual Recognition Challenge)

The classification task on ImageNet:

For each image, assign 5 labels in order of decreasing confidence.
one of these labels matches the ground truth

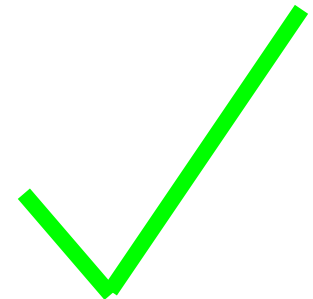
Success if



https://commons.wikimedia.org/wiki/File:Common_zebra_1.jpg

Predictions:

1. Carpet
2. Zebra
3. Llama
4. Flower
5. Horse



ILSVRC 2012

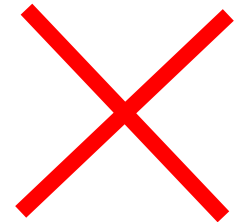
Percentage that model fails to classify is known as ***Top 5 Error Rate***



https://commons.wikimedia.org/wiki/File:Puffer_Fish_DSC01257.JPG

Predictions:

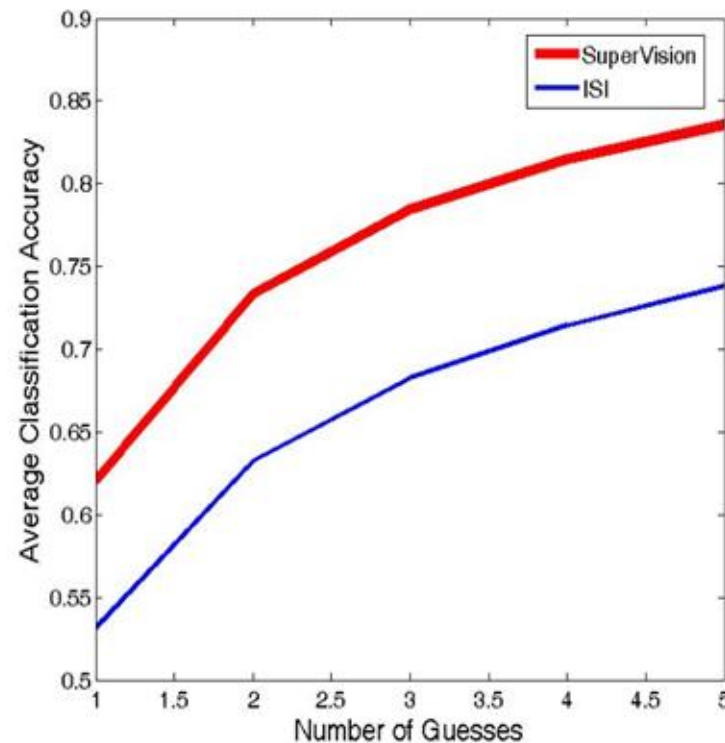
1. Sponge
2. Person
3. Llama
4. Flower
5. Boat



AlexNet: Why CNNs Are a Big Deal

Major performance boost on ImageNet at ILSVRC 2012

Top 5 error rate of 15.3% compared to 26.2% achieved by 2nd place

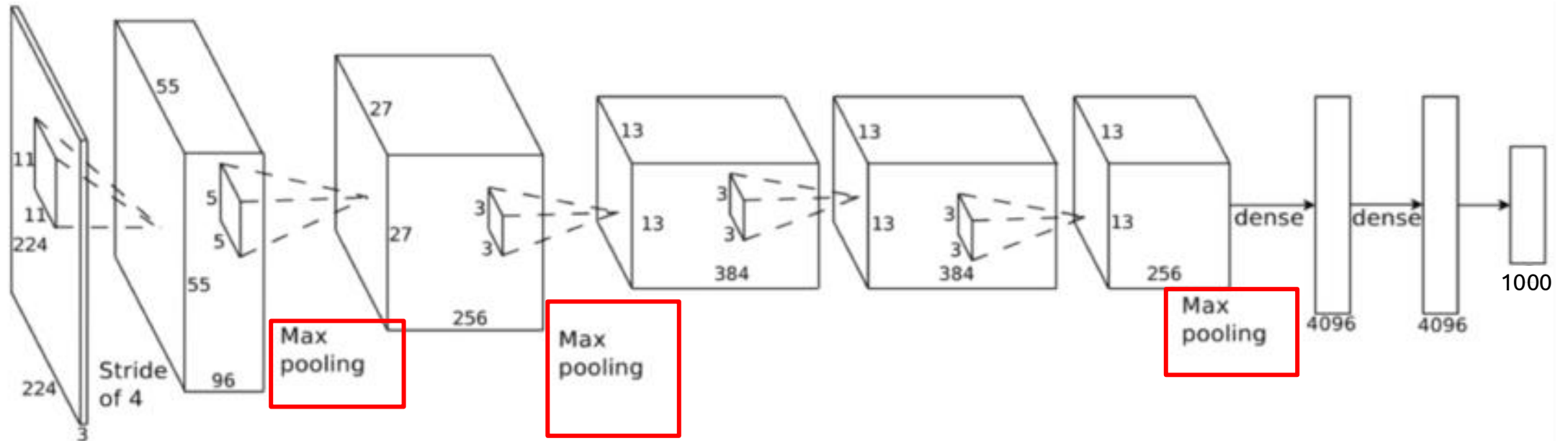


Note: SuperVision is the name of Alex's team

<http://image-net.org/challenges/LSVRC/2012/analysis/>

AlexNet

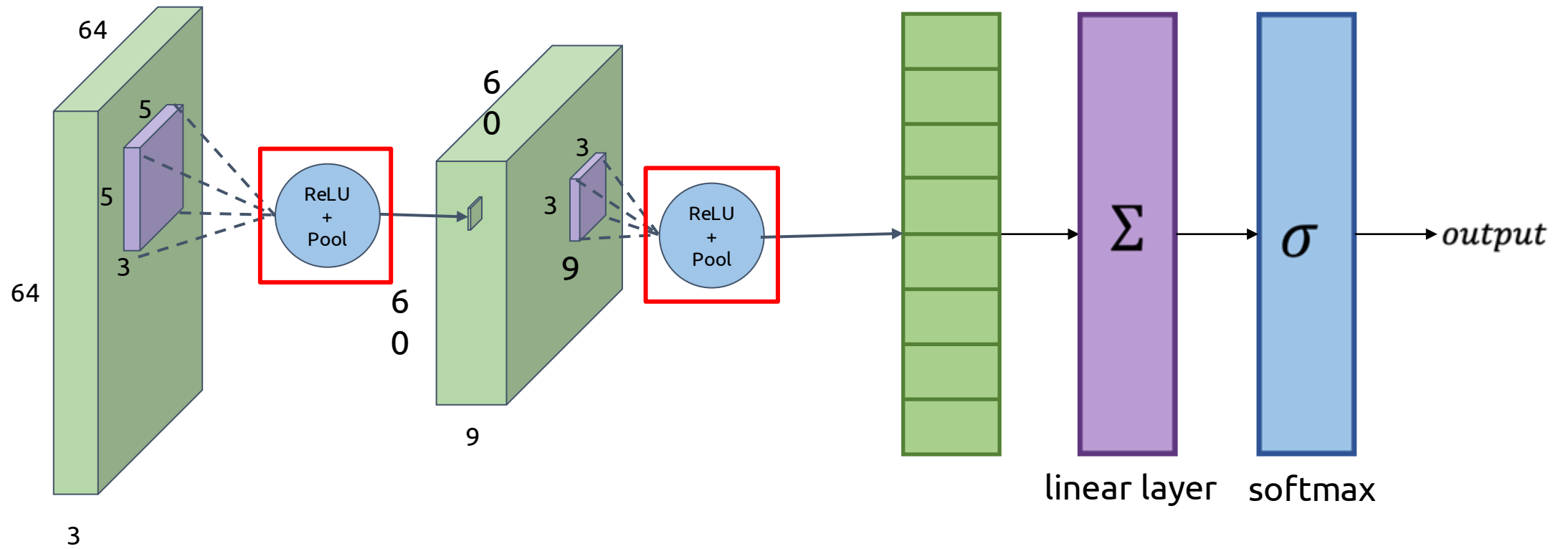
- 60 million parameters
- 5 Convolutional Layers
- 3 Fully Connected Layers



[Alex Krizhevsky et al. 2012]

<https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>

Pooling



Max Pooling

Max pooling with stride 2 and 2x2 filters

6	3	1	-3
4	1	2	0
3	1	3	2
7	1	1	1

Max of pixels
in window



Max Pooling

Max pooling with stride 2 and 2x2 filters

6	3	1	-3
4	1	2	0
3	1	3	2
7	1	1	1

Max of pixels
in window



6	

Max Pooling

Max pooling with stride 2 and 2x2 filters

6	3	1	-3
4	1	2	0
3	1	3	2
7	1	1	1

Max of pixels
in window



6	2

Max Pooling

Max pooling with stride 2 and 2x2 filters

6	3	1	-3
4	1	2	0
3	1	3	2
7	1	1	1

Max of pixels
in window



6	2
7	

Max Pooling

Max pooling with stride 2 and 2x2 filters

6	3	1	-3
4	1	2	0
3	1	3	2
7	1	1	1

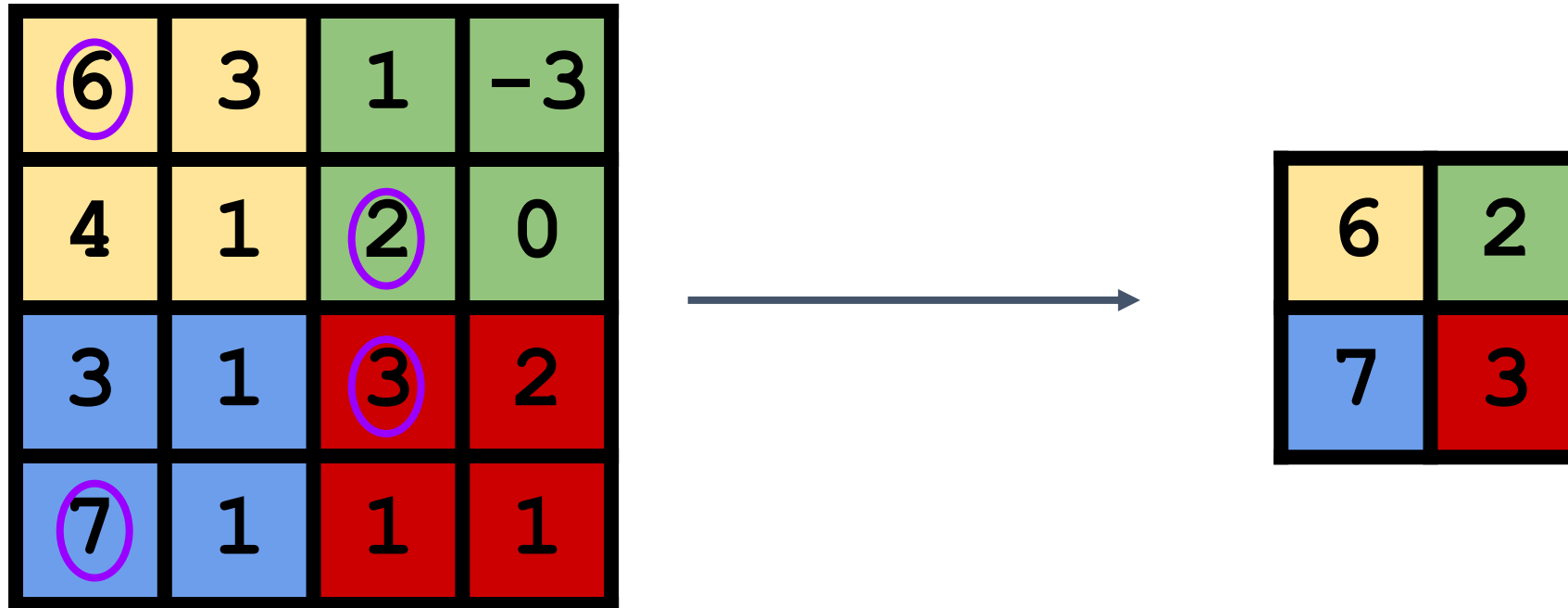
Max of pixels
in window



6	2
7	3

Max Pooling

Max pooling with stride 2 and 2x2 filters



Why use Max Pooling?



Pooling: Motivation

Max Pooling

- Keeps track of regions with highest activations, indicating object presence
- Controllable way to lower (coarser) resolution (down sample the convolution output)



Original Image



Convolution Output



After Pooling

Convolutional Neural Network Demo

Regularization in deep learning

Overfitting: What can we do about it?

1. Early stopping

Early stopping: pseudocode

```
curr_valid_loss = inf
for i in range(n_epochs):
    train model()
    new_valid_loss = model.get_test_loss()
    if new_valid_loss > curr_valid_loss:
        break
    else:
        curr_valid_loss = new_valid_loss
```

Early stopping: thoughts

This is kind of a hack...

Can we stop validation loss from increasing altogether?

Any ideas?

Overfitting: What can we do about it?

1. Early stopping
- 2. Reduce parameters**

Reduce parameters - why?

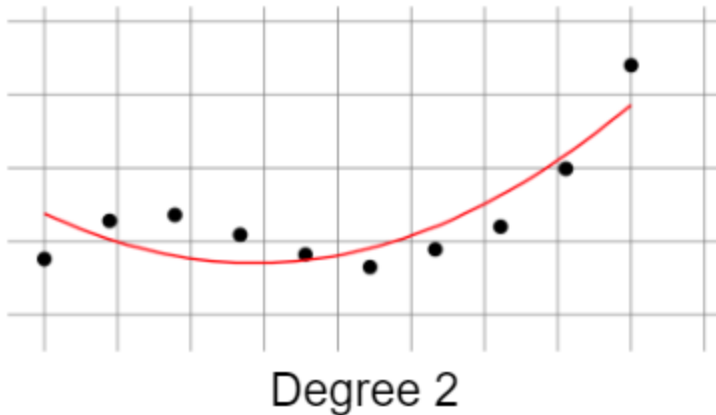
More parameters = more 'knobs' to fiddle = more possibilities to learn something overly-specific to the training data

- Example: curve fitting

Reduce parameters - why?

More parameters = more 'knobs' to fiddle = more possibilities to learn something overly-specific to the training data

- Example: curve fitting

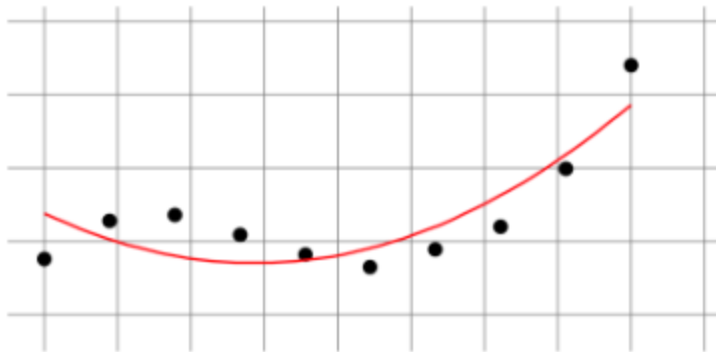


$$y = 3x^2 + 2x + 4$$

Reduce parameters - why?

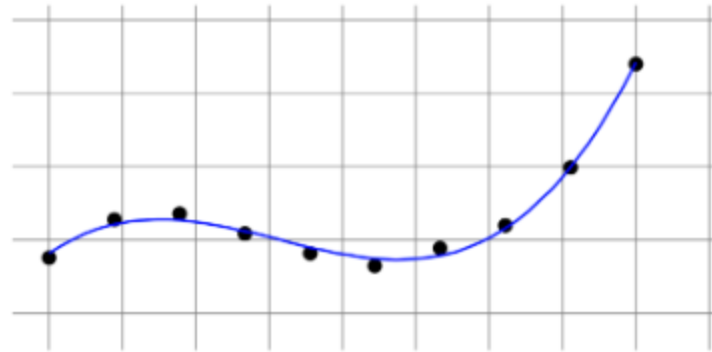
More parameters = more 'knobs' to fiddle = more possibilities to learn something overly-specific to the training data

- Example: curve fitting



Degree 2

$$y = 3x^2 + 2x + 4$$



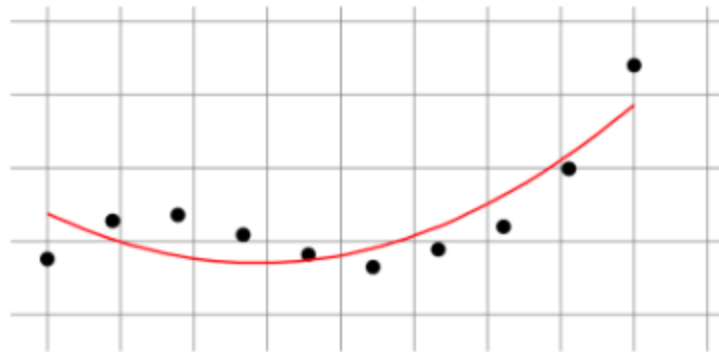
Degree 3

$$y = 2x^3 - 4x^2 + x - 3$$

Reduce parameters - why?

More parameters = more 'knobs' to fiddle = more possibilities to learn something overly-specific to the training data

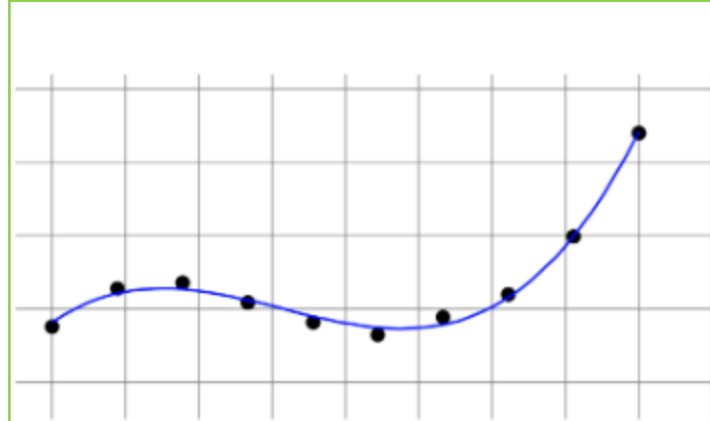
- Example: curve fitting



Degree 2

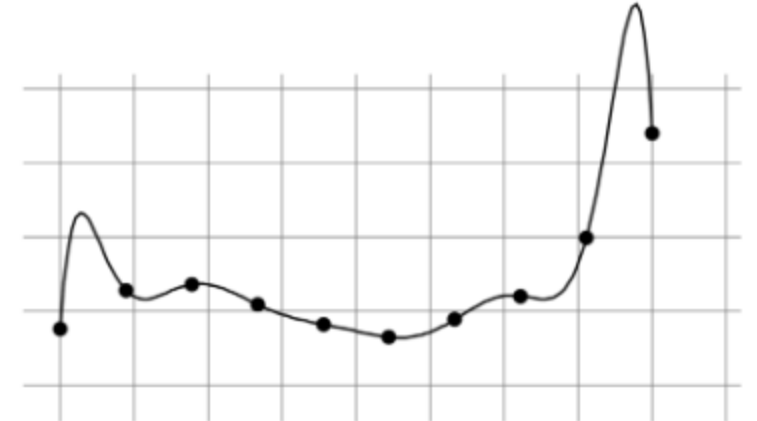
$$y = 3x^2 + 2x + 4$$

This is probably the best fit curve...



Degree 3

$$y = 2x^3 - 4x^2 + x - 3$$



Degree 10

$$\begin{aligned} y &= 7x^{10} + x^9 + 6x^8 - 3x^7 \\ &\quad - 2x^6 \dots + x - 3 \end{aligned}$$

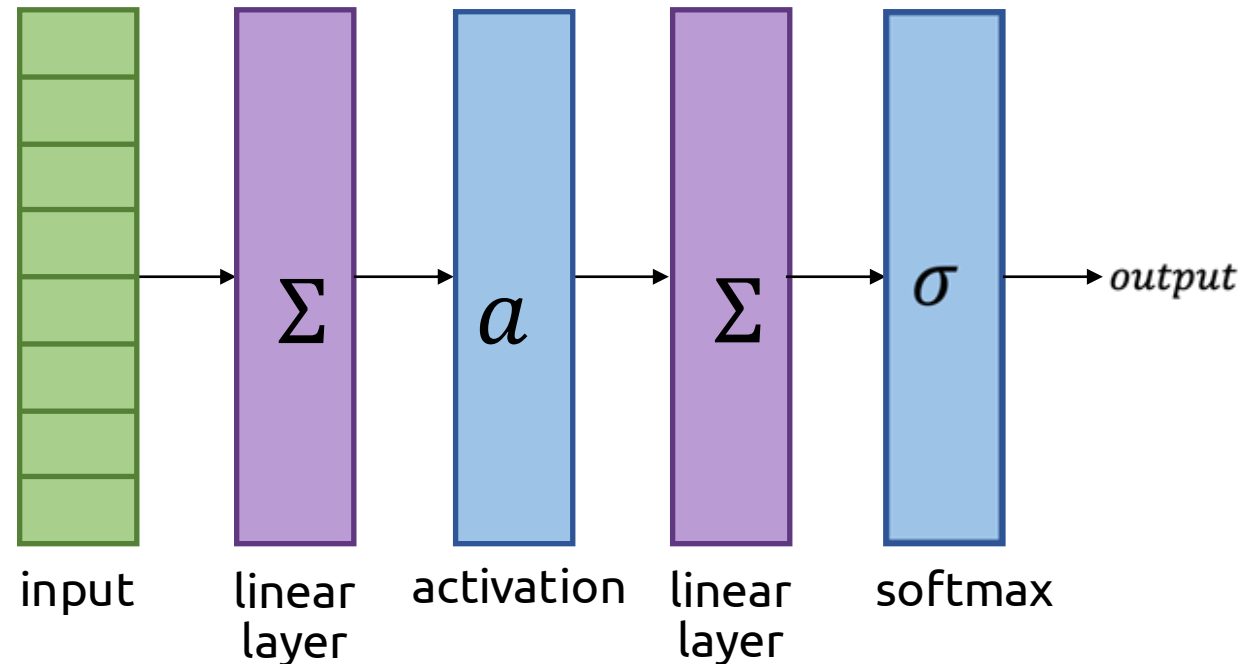
Reduce parameters in a Neural Net: How?

- Fully connected nets?
- CNN?
- ***Any*** NN?

Reduce parameters in a Neural Net: How?

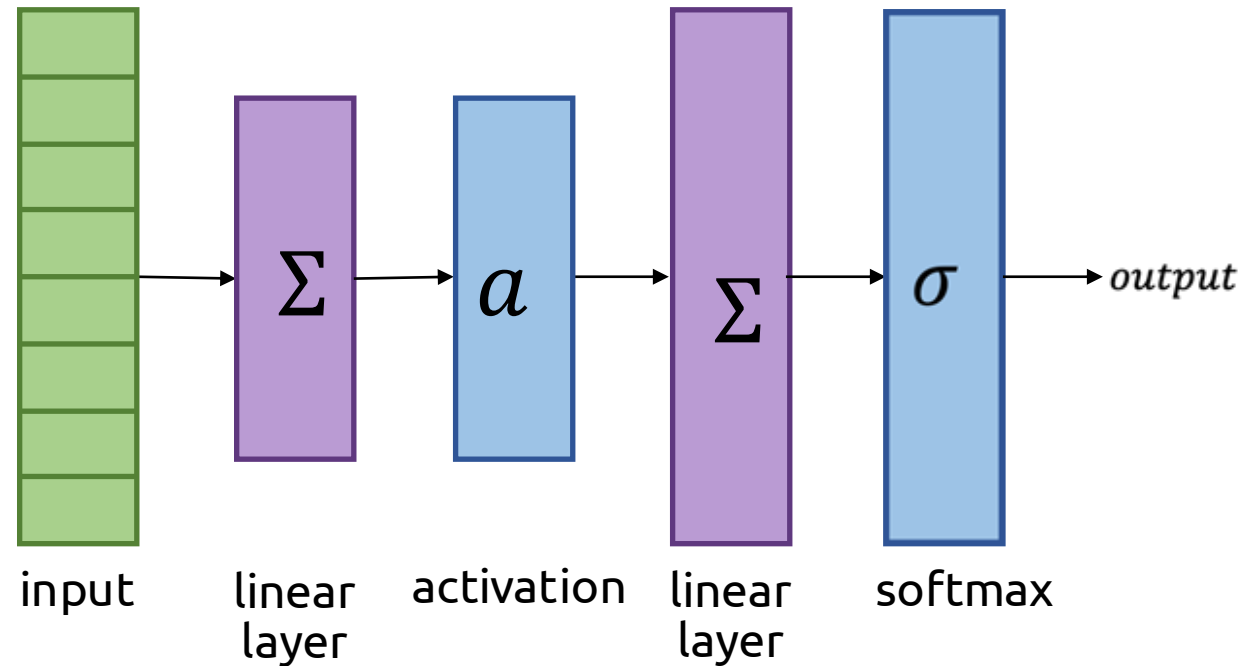
- Fully connected:

What can be done here?



Reduce parameters in a Neural Net: How?

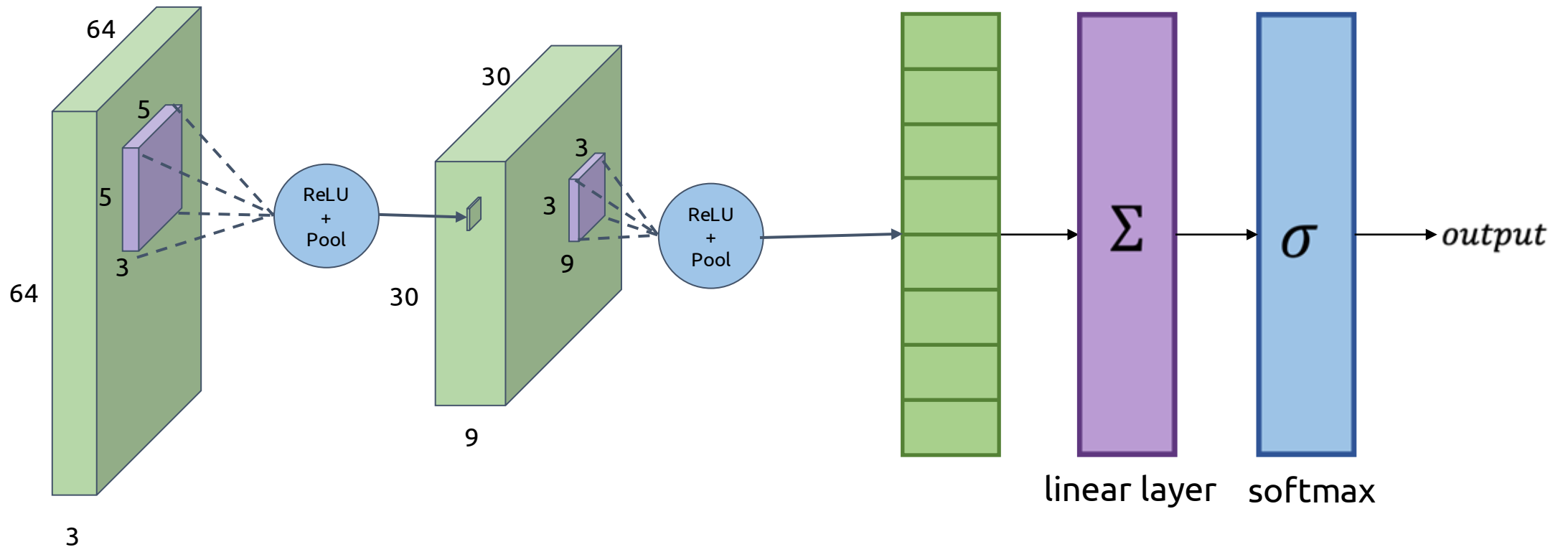
- Fully connected: reduce layer size



Reduce parameters in a Neural Net: How?

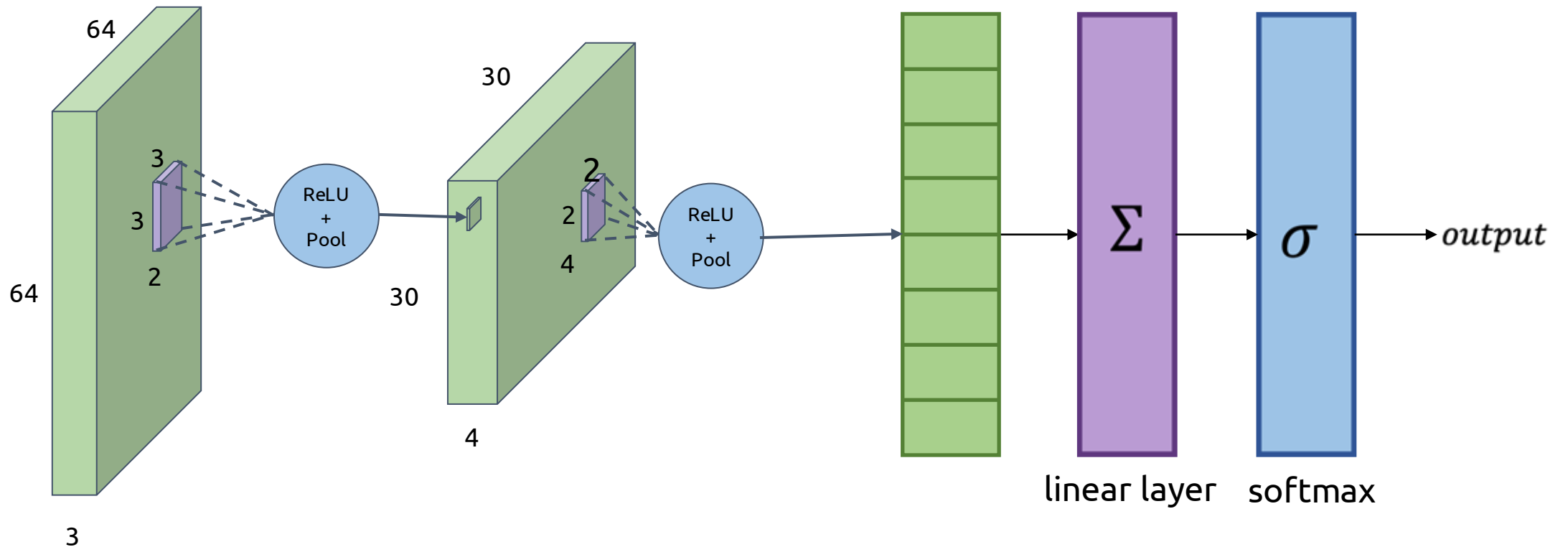
- CNN:

What about here?



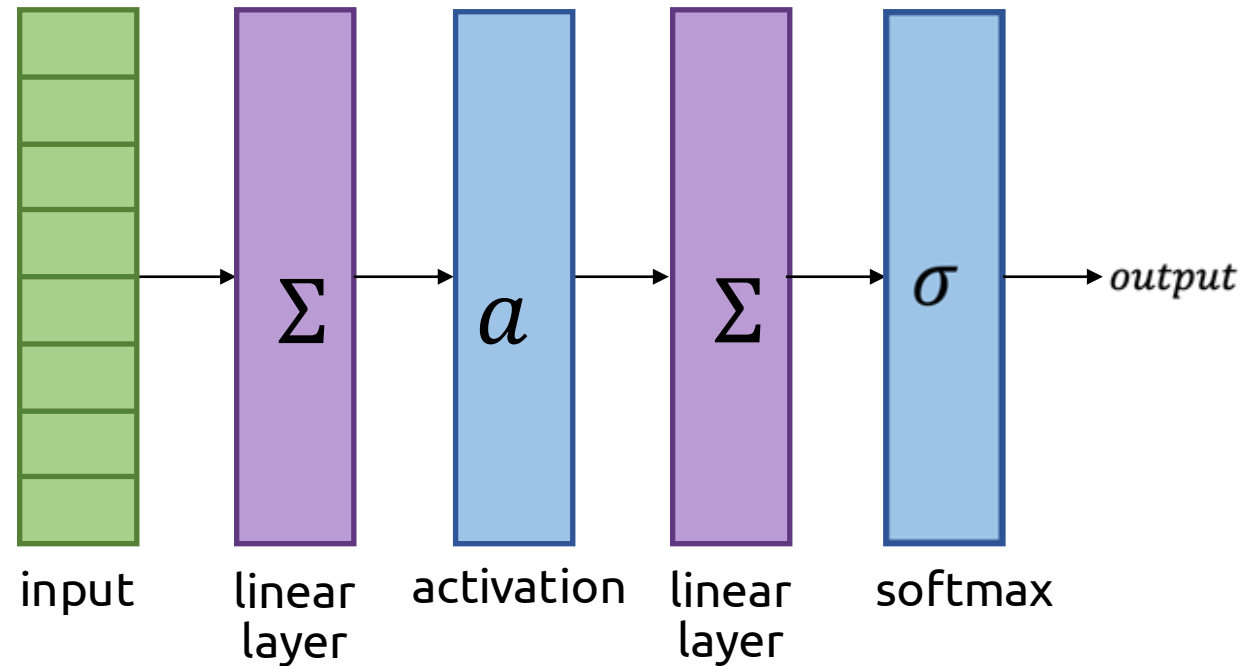
Reduce parameters in a Neural Net: How?

- CNN: decrease num_channels (and also possibly filter_size)



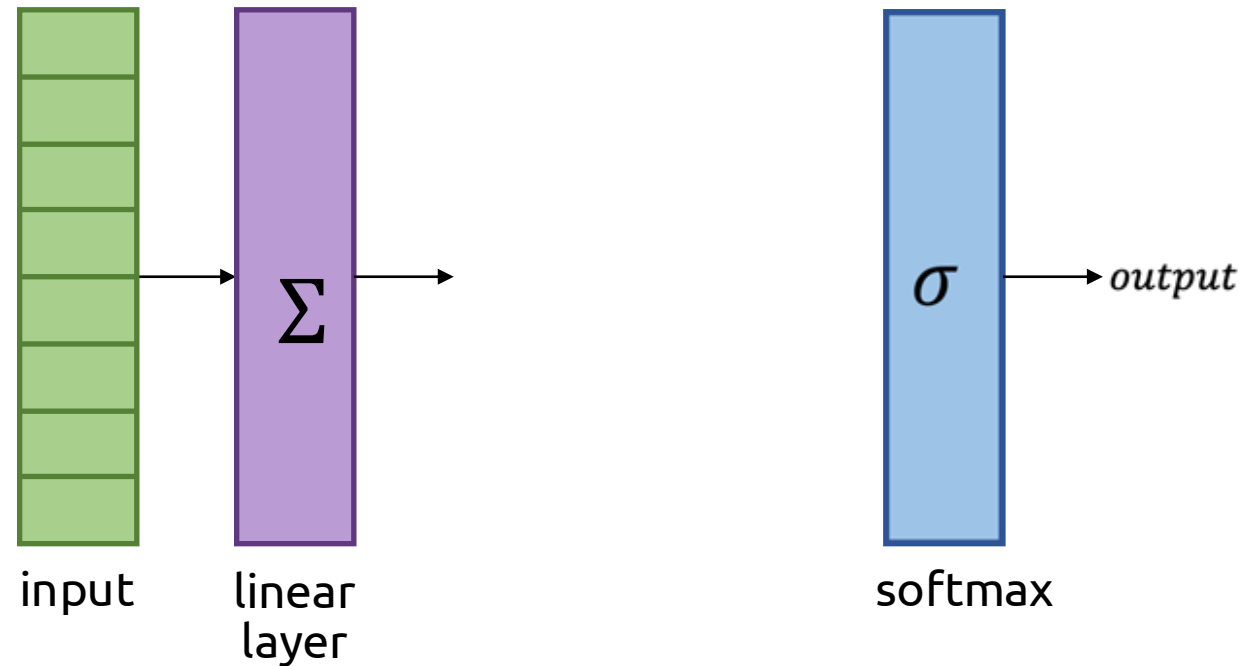
Reduce parameters in a Neural Net: How?

- All architectures: Decrease number of layers



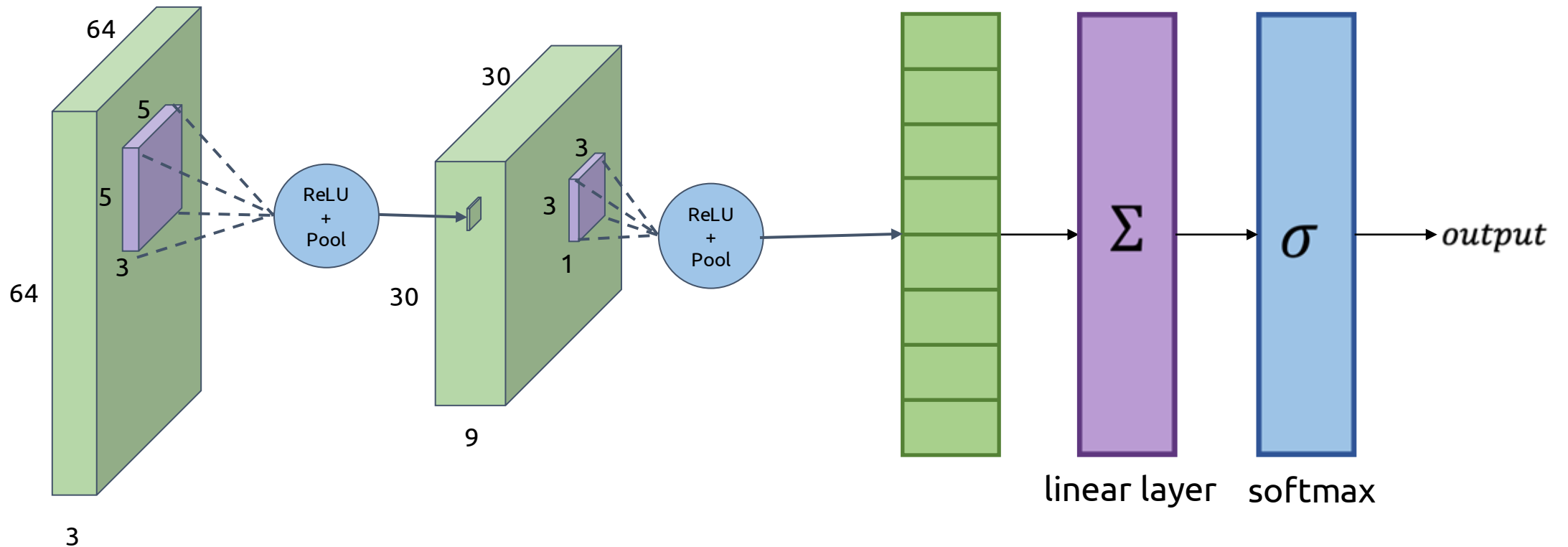
Reduce parameters in a Neural Net: How?

- All architectures: Decrease number of layers



Reduce parameters in a Neural Net: How?

- All architectures: Decrease number of layers

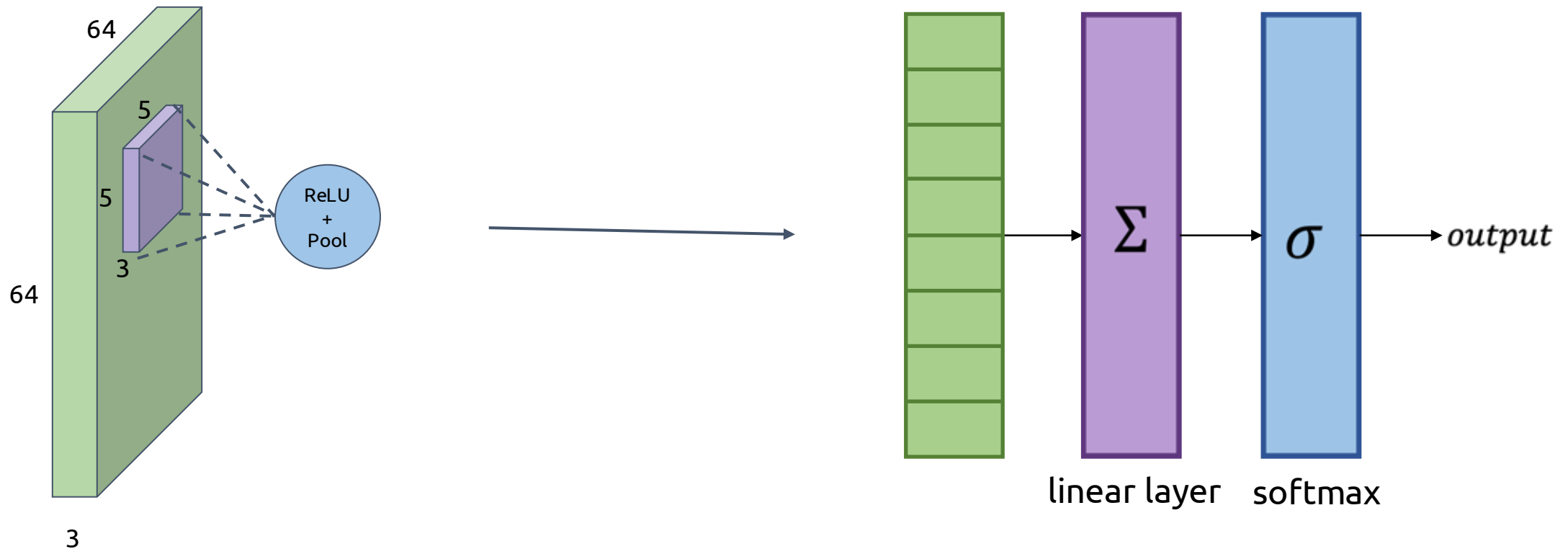


Any questions?



Reduce parameters in a Neural Net: How?

- All architectures: Decrease number of layers



Reducing parameters: Why not?

- Seen (by deep learning folks) as a bit 'old-fashioned'
 - Classical perspective: model complexity should match data complexity
 - Deep learning perspective: all real-world data is infinitely complex (i.e. there are infinite variations on what a handwritten digit can look like). If your model is overfitting, that just means you don't have enough data—so get more!
 - What if we can't get more data?
 - (e.g. it's prohibitively expensive to gather more data)
- Synthesize*** variations on your data

Overfitting: What can we do about it?

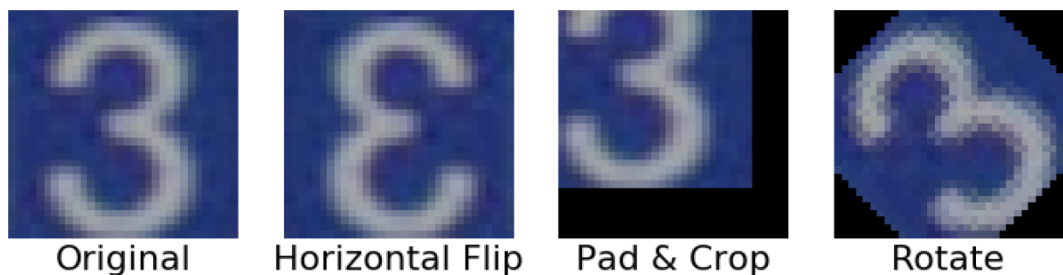
1. Early stopping
2. Reduce parameters
- 3. Data augmentation**

Data Augmentation

- Generate (random) variations on your training data, treat that as more training data
 - Assumption: your (random) variations still produce data that is within the expected distribution of training data (so you can't get **too** crazy)
- Most commonly used on image data

Data Augmentation: Image Examples

Geometric Transformations



https://bair.berkeley.edu/blog/2019/06/07/data_aug/

Fancy (Learned) Semantic Transforms (i.e. “Image synthesis for data augmentation”)



<https://junyanz.github.io/CycleGAN/>

Filters & Pixel Intensity Transforms Blur, sharpen, contrast adjust, ...



<https://towardsdatascience.com/data-augmentation-for-deep-learning-4fe21d1a4eb9>

Data Augmentation: Limitations

- Your model might still overfit!
 - In general, it's impossible to design an augmentation procedure that covers ***all*** the dimensions of variation your data might experience
 - Your model can still overfit to patterns in the dimensions that you don't augment with variations...

Overfitting: What can we do about it?

1. Early stopping
2. Reduce parameters
3. Data augmentation
- 4. Dropout**

Dropout – general intuition

- Preventing the network from learning under perfect conditions; that is, make it **harder** for the network to learn

A climbing analogy:

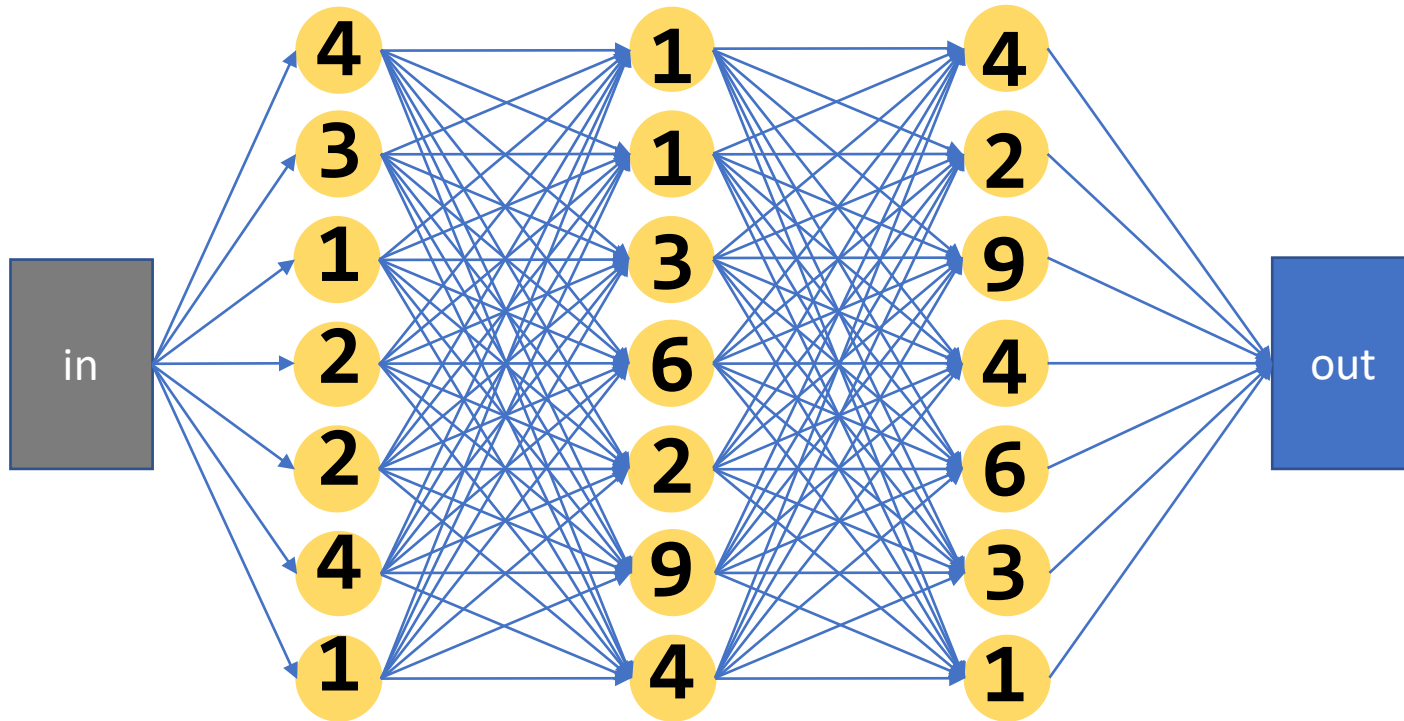
A person is climbing a wall using holds

- What if, I make a rule that she can climb
- ... only using certain holds (say just green ones!)
- If she can learn to do this using fewer holds...
- ...she'll definitely be able to do it with ALL the holds
- (learn better climbing techniques in the process)

Dropout $\sim$ using only a certain holds instead of ALL the holds

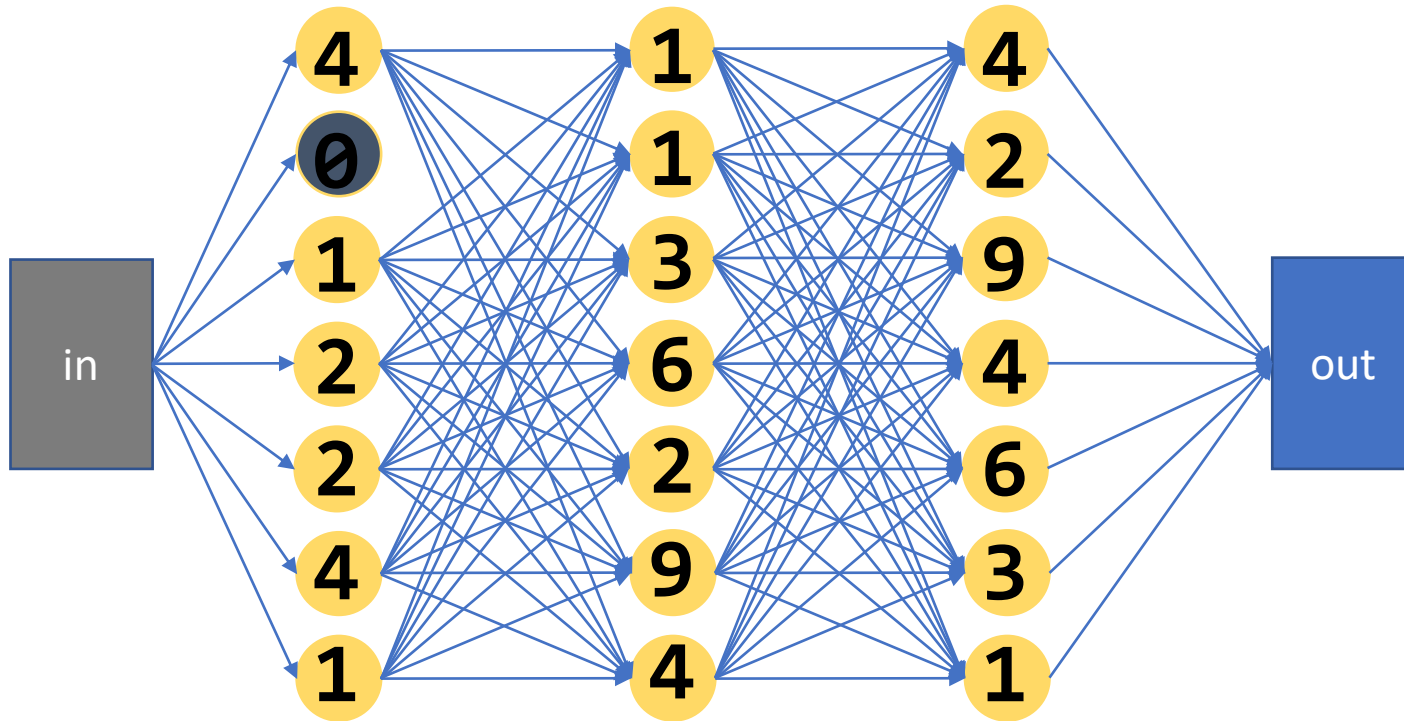


Dropout - what?



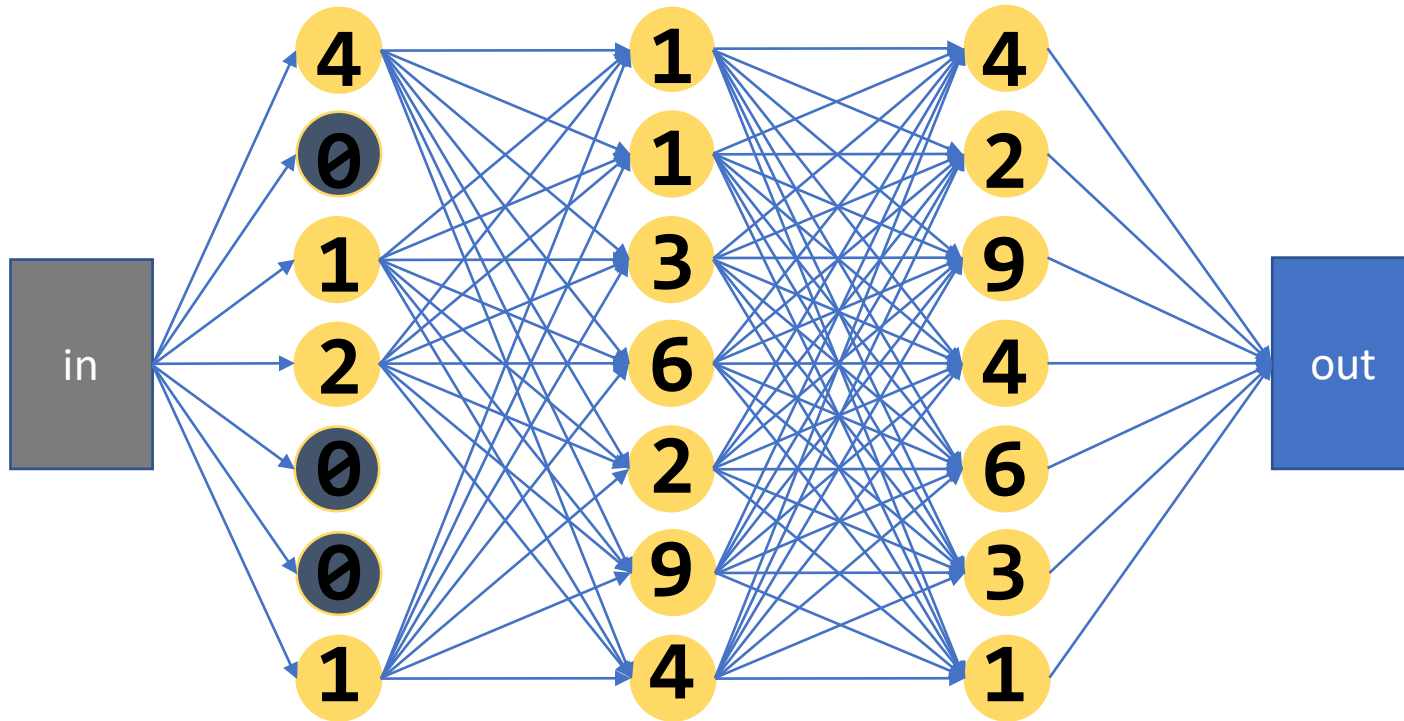
Typical NN: the output of every node in every layer is used in the next layer of the network

Dropout - what?



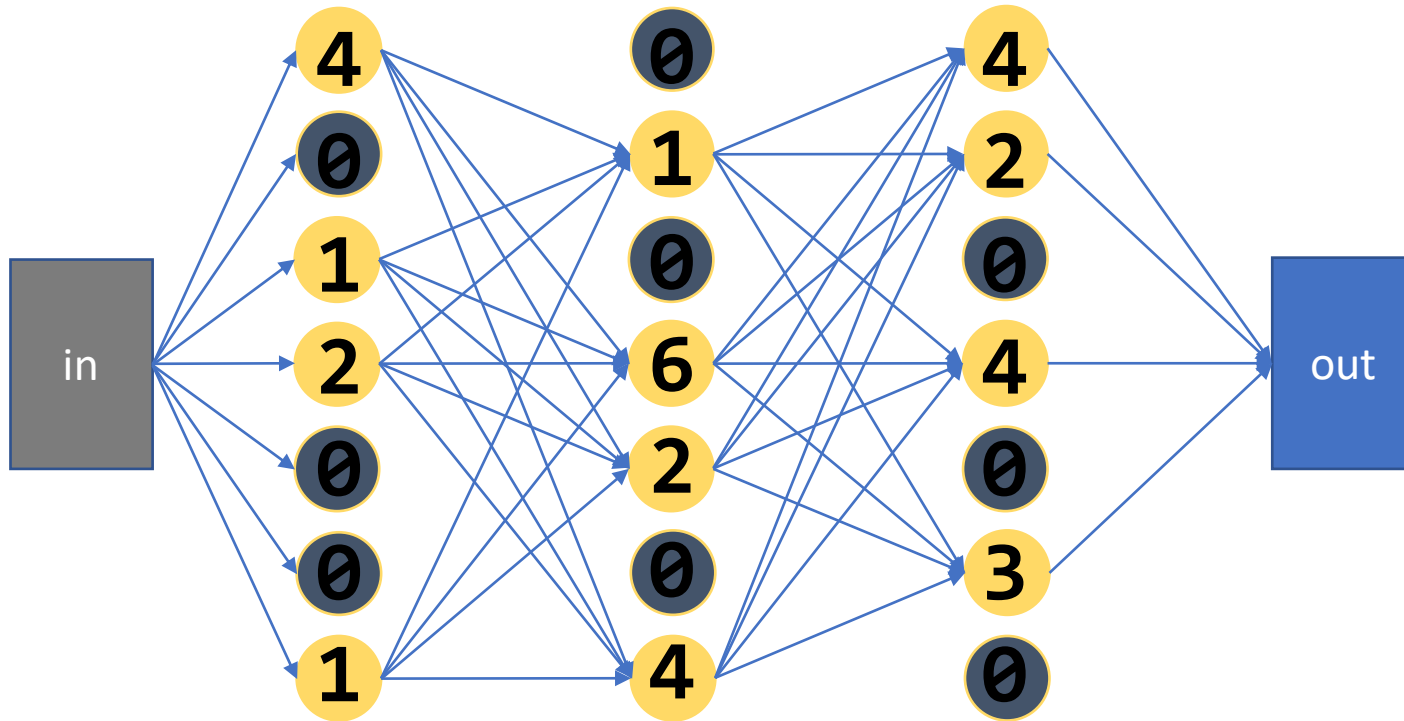
Dropout: *in a single training pass*, the output of randomly selected nodes from each layer will “drop out”, i.e. be set to 0

Dropout - what?



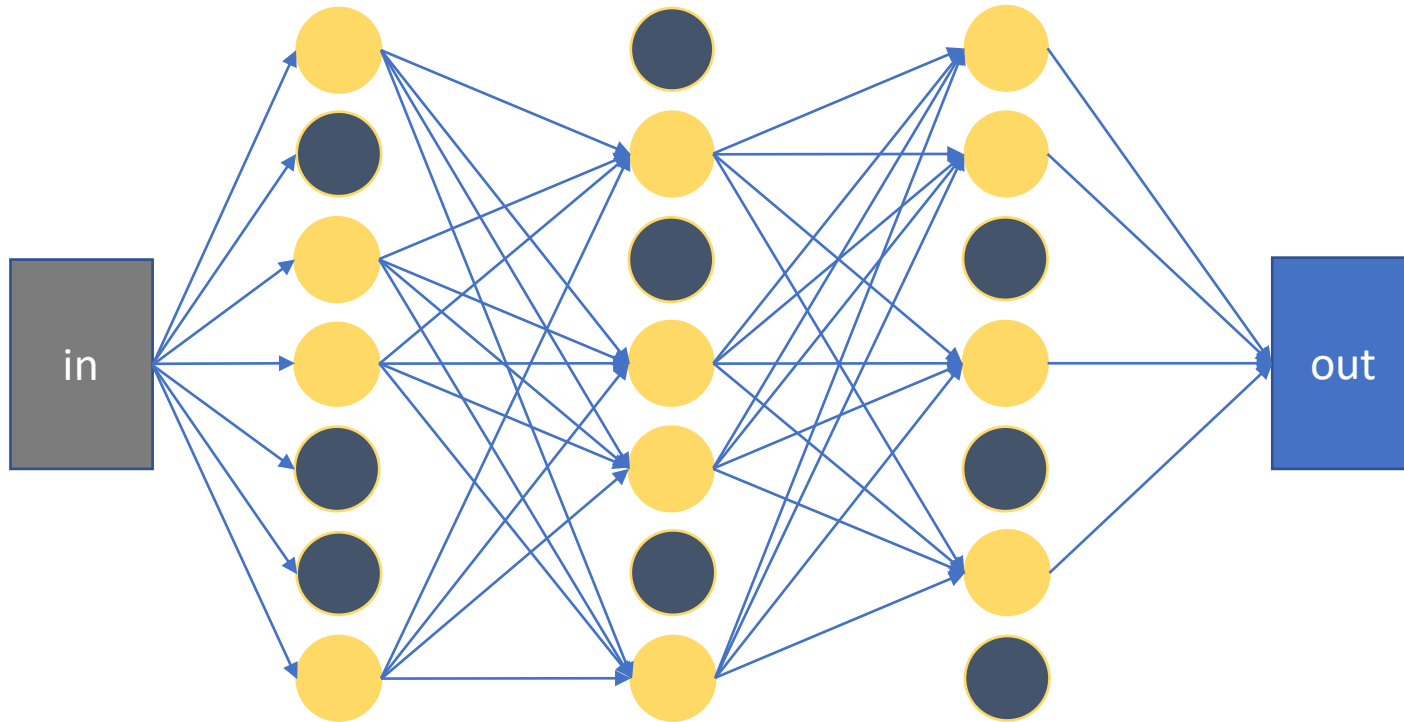
Dropout: *in a single training pass*, the output of randomly selected nodes from each layer will “drop out”, i.e. be set to 0

Dropout - what?



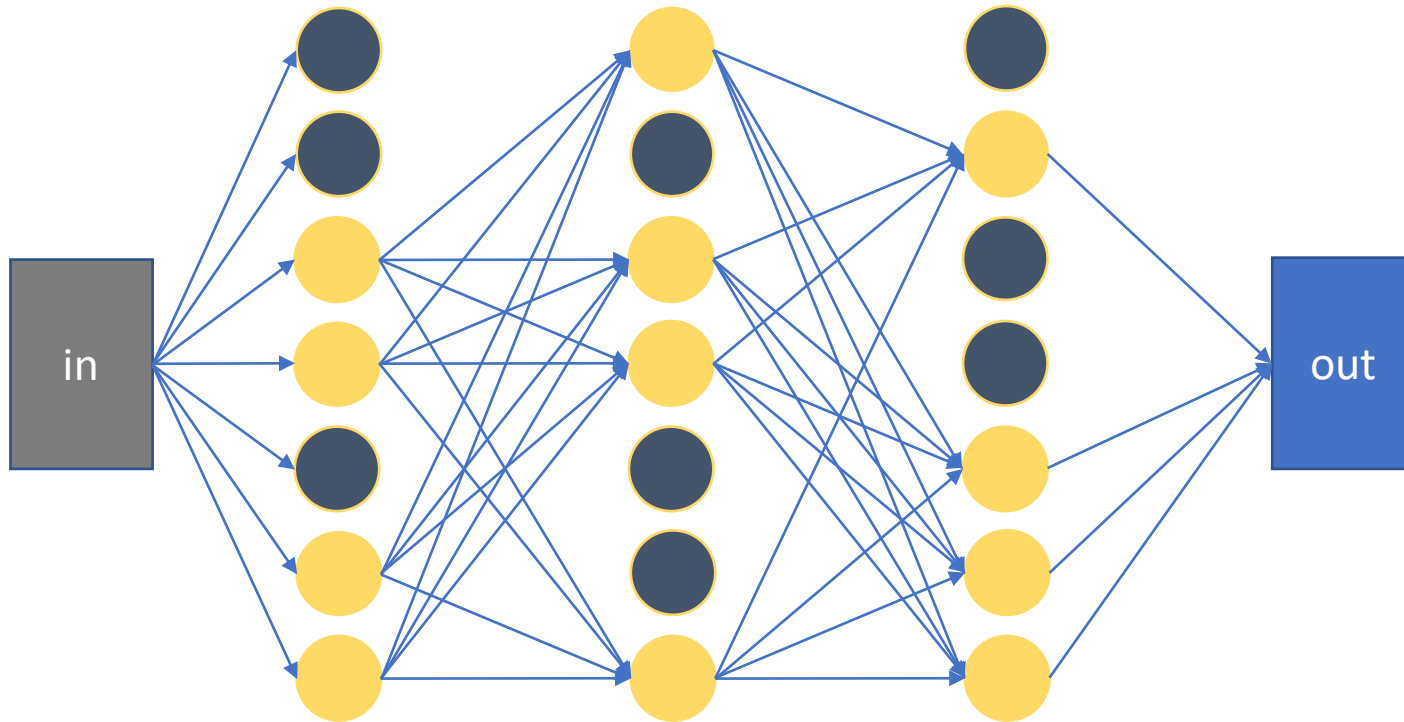
Not just limited to the input layer: can do this to *any* layer of the network

Dropout - what?



The nodes that drop out will be different each pass (re-randomly selected)

Dropout - what?



The nodes that drop out will be different each pass (re-randomly selected)

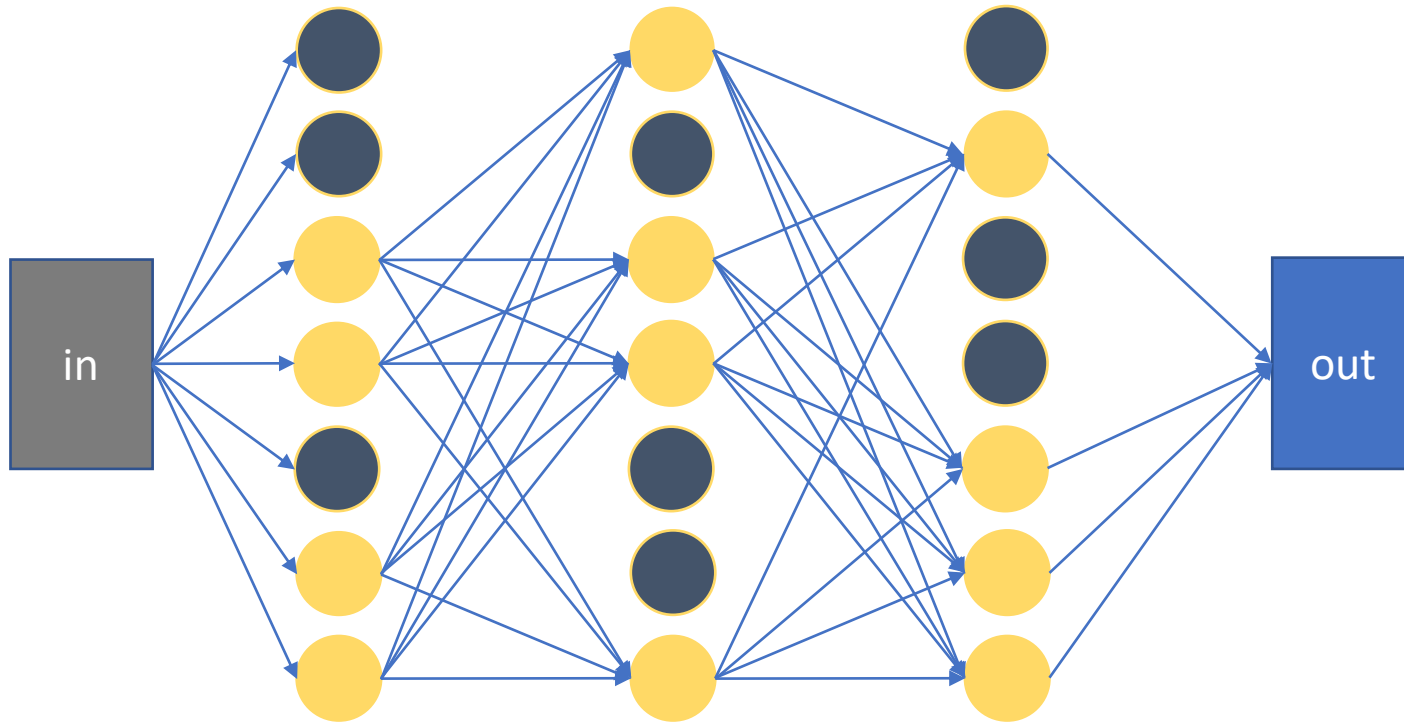
Dropout - why?

- Sort of looks like data augmentation, if you squint hard enough
 - Augmenting the data by randomly dropping out parts of it
- Over multiple passes through the net (i.e. during training over many epochs):
 - Randomly dropping neurons “forces” each neuron to learn a non-trivial weight
 - The network can’t learn to rely on spurious correlations (i.e. meaningless patterns), because they randomly might not be present

Do we use dropout while testing?

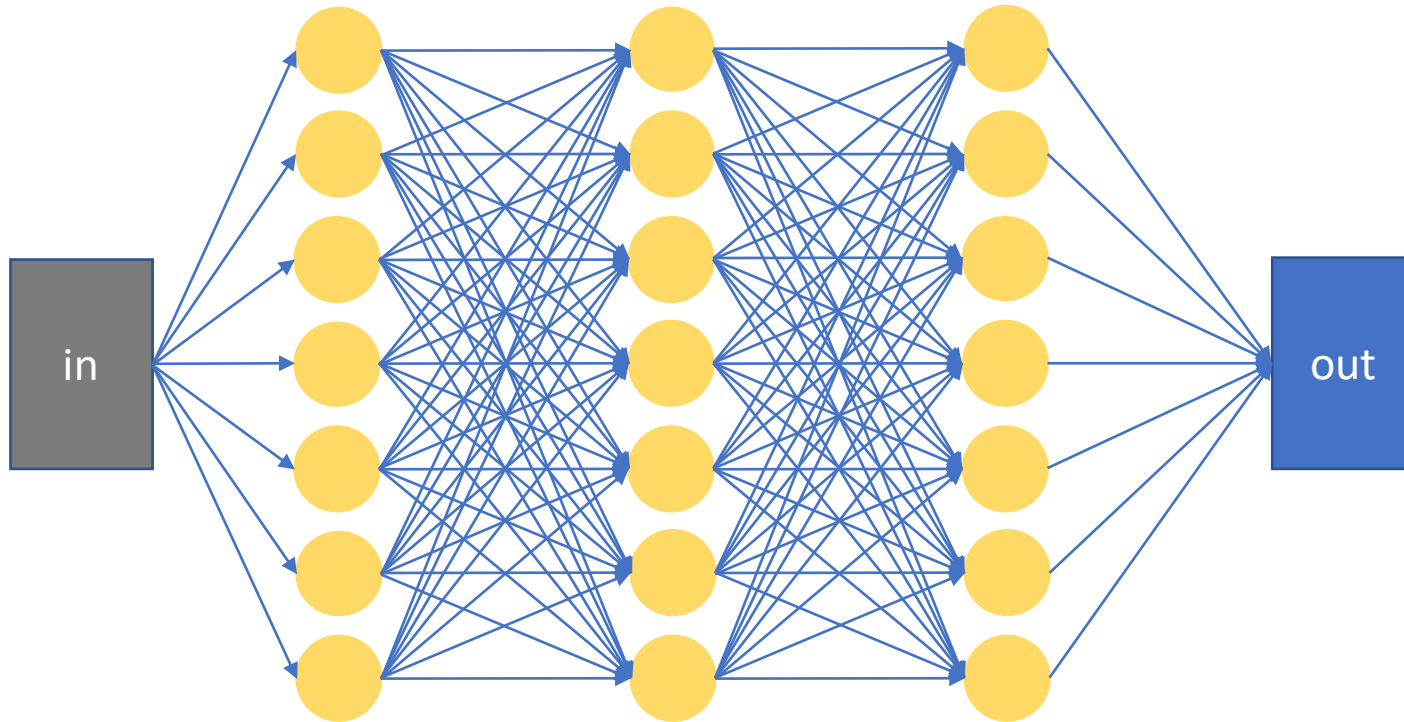
Why not?

Dropout: Implications for test time



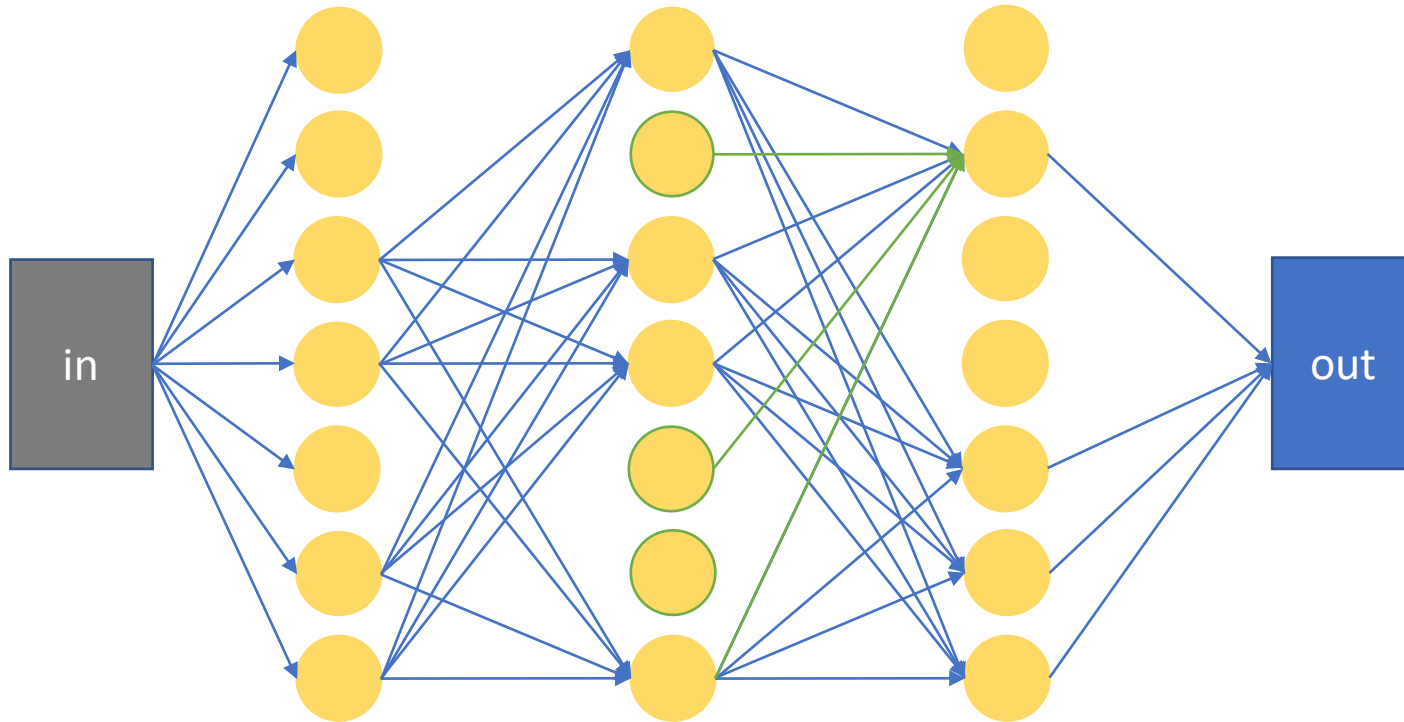
- During testing, we stop dropping out and use all of the neurons again

Dropout: Implications for test time



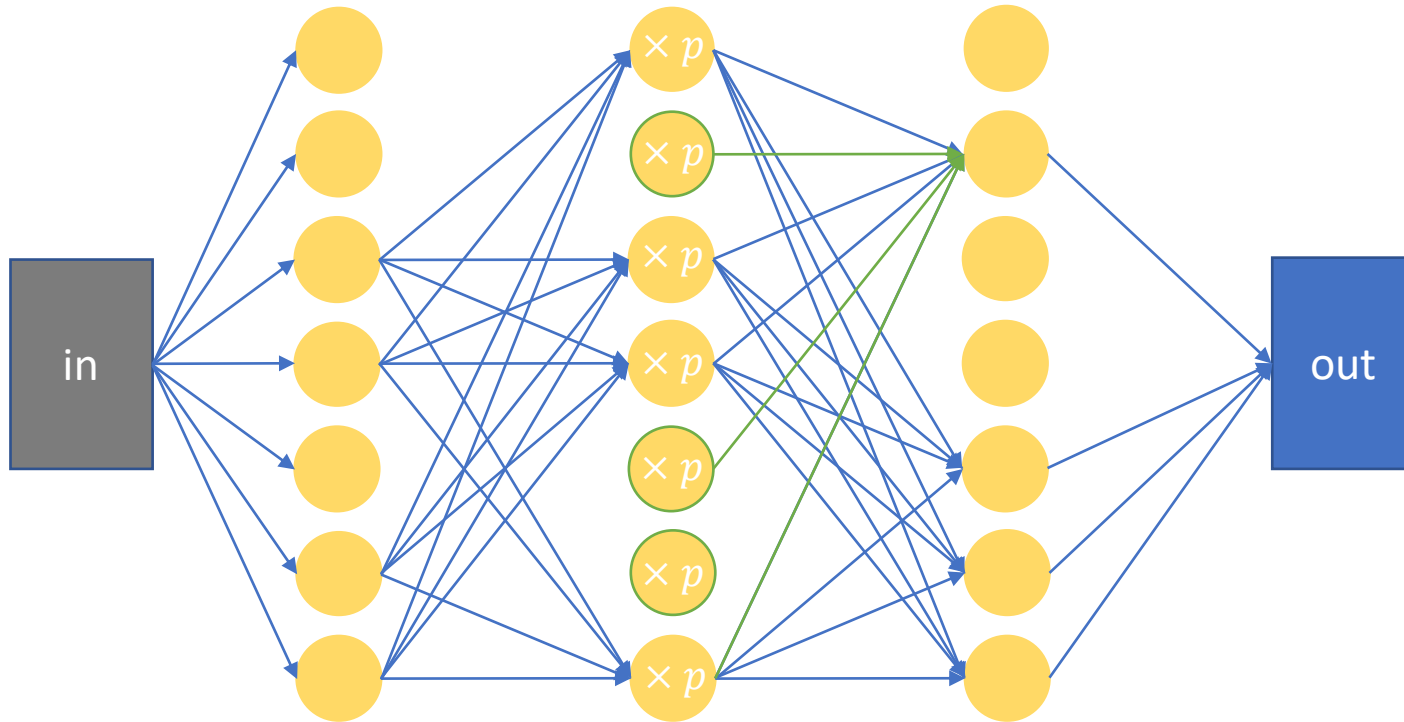
- During testing, we stop dropping out and use all of the neurons again

Dropout: Implications for test time



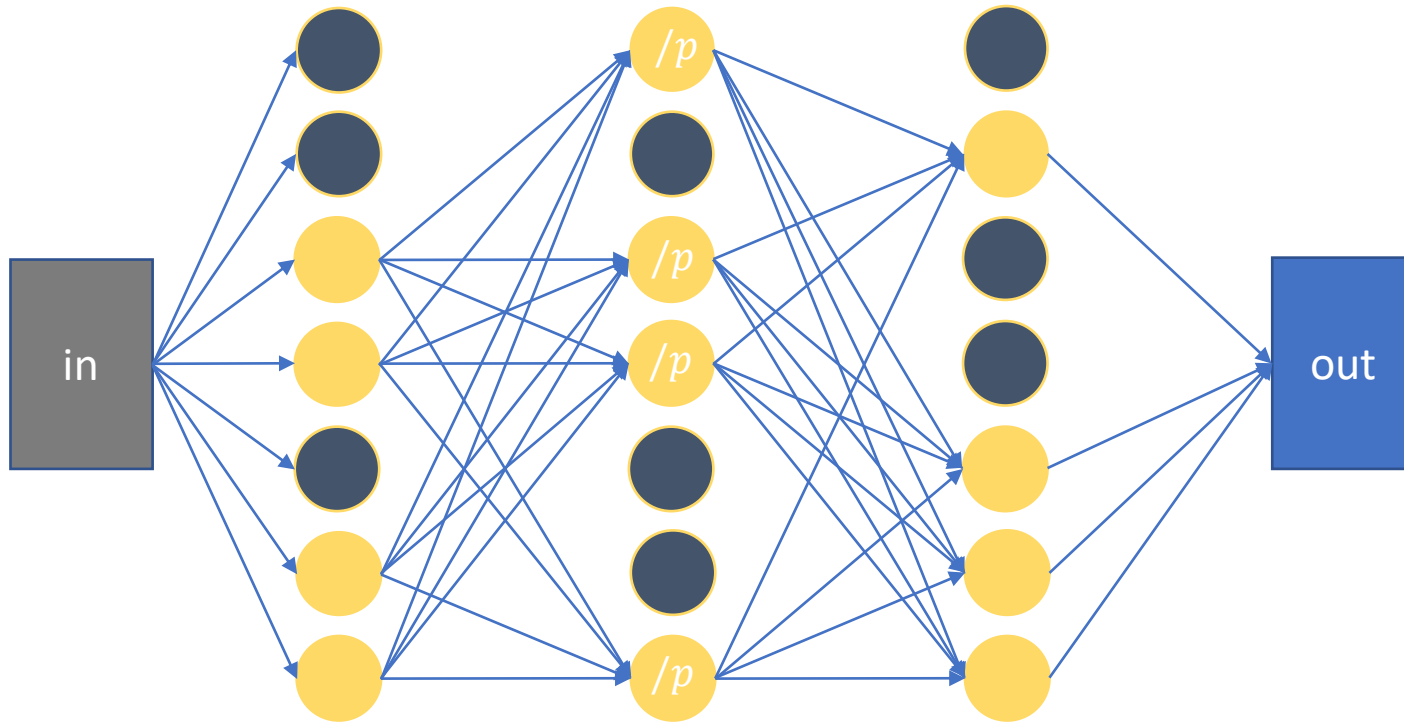
- During testing, we stop dropping out and use all of the neurons again
- If a layer keeps a fraction p of its neurons during training, then when we use all the neurons at test time, the next layer will get a bigger input than expected...
- ***What do we do!?***

Dropout: Implications for test time



- **Solution 1:**
Multiply the values of all neurons by p , so that the expected magnitude of the sum of neurons is the same

Dropout: Implications for test time



- **Solution 1:**
Multiply the values of all neurons by p , so that the expected magnitude of the sum of neurons is the same
- **Solution 2:**
At training time, divide the values of the kept neurons by p

Dropout - implementation

Any questions?



```
nn.Dropout (p=0.1)
```

- Hyperparameter **p** between $[0, 1]$: the rate at which the outputs of the previous layer are dropped
- **p** = **0.5**: drop half, keep half
- **p** = **0.25**: drop $\frac{1}{4}$, keep $\frac{3}{4}$

Putting it all together

[Convolutional Neural Network Demo](#)

TL;DR – Rule of Thumb for Overfitting:

“Make start with the small-ish architecture and your net bigger until it starts to overfit...

(use train/validation loss curves to monitor)

...then apply one of the techniques from this lecture”

Recap – Learn about activations and Convolutional Neural Networks (CNNs)

(1) Convolution in Pytorch

(2) Building our CNN model

(3) Maxpooling

(4) Regularization for deep learning models

(5) Class activity: CNN and regularization demo

Homework: Finish [Chapter 7](#) of the Dive into Deep Learning Book

Wrap up



What was the clearest point today?

What was the muddiest point today?



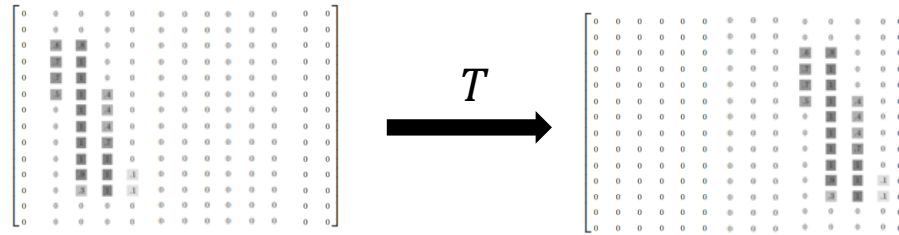
Extra slides

So...did we achieve our goal of translational invariance?



What was Translational Invariance again?

- To make a neural net f robust in this same way, it should ideally satisfy ***translational invariance***: $f(T(x)) = f(x)$, where
 - x is the input image
 - T is a translation (i.e. a horizontal and/or vertical shift)



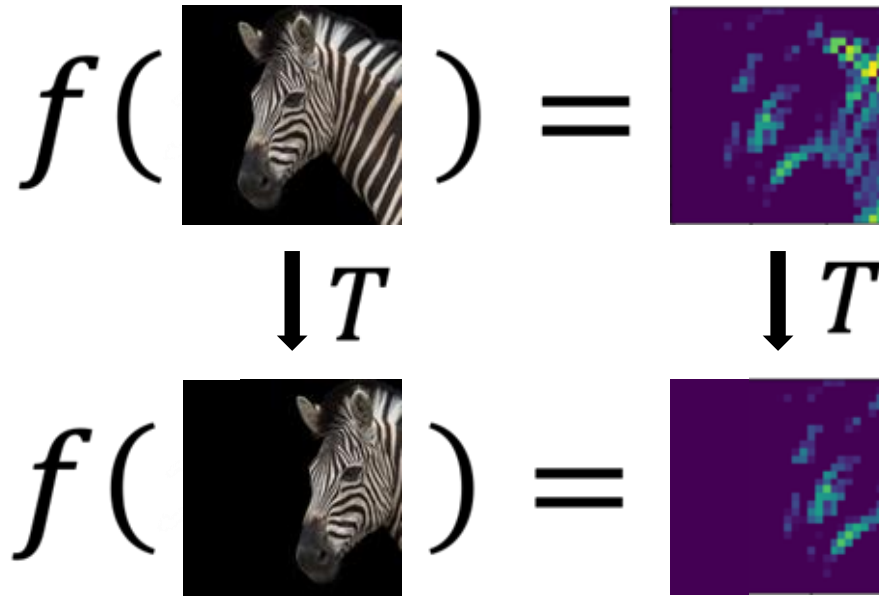
[illegible]

Are CNNs Translation Invariant?

- Convolution is ***translation equivariant***
 - A translated input results in an output translated by the same amount

- $f(T(I)) = T(f(I))$

- $(T(I) \otimes K)(x, y) = T(I \otimes K)(x, y)$



* Here, $(I \otimes K)(x, y) = \sum_m \sum_n I(x + m, y + n) K(m, n)$

Are CNNs Translation Invariant?

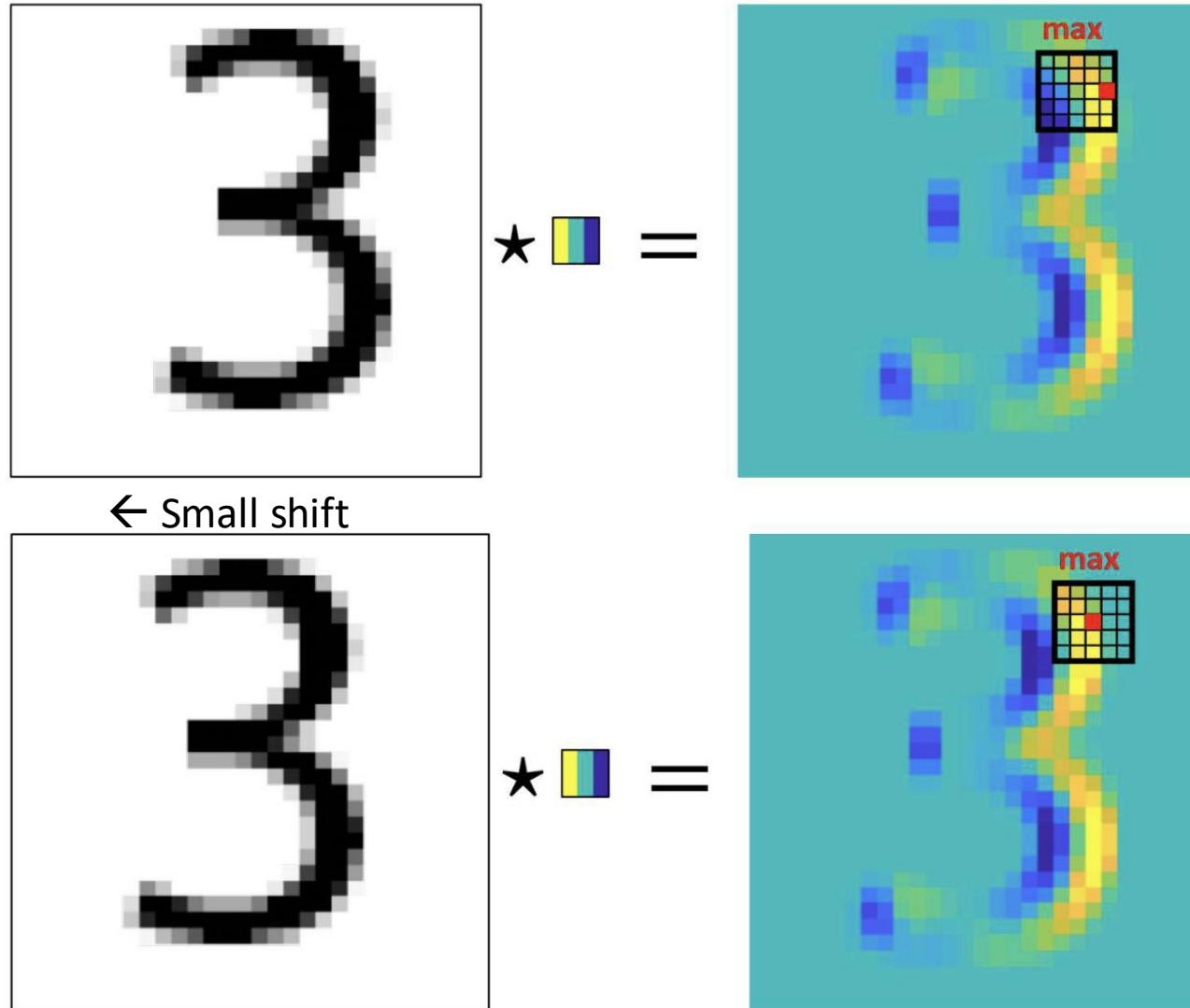
- Max pooling is intended to give invariance to small translations
 - The highest activation pixel can shift around within the pooling window, and the output does not change

$$f\left(\begin{array}{|c|c|}\hline 6 & 3 \\ \hline 4 & 1 \\ \hline\end{array}\right) = 6$$

$$f\left(\begin{array}{|c|c|}\hline 1 & 5 \\ \hline 6 & 3 \\ \hline\end{array}\right) = 6$$

$$f\left(\begin{array}{|c|c|}\hline 2 & 6 \\ \hline 2 & 4 \\ \hline\end{array}\right) = 6$$

So how does it all come together?

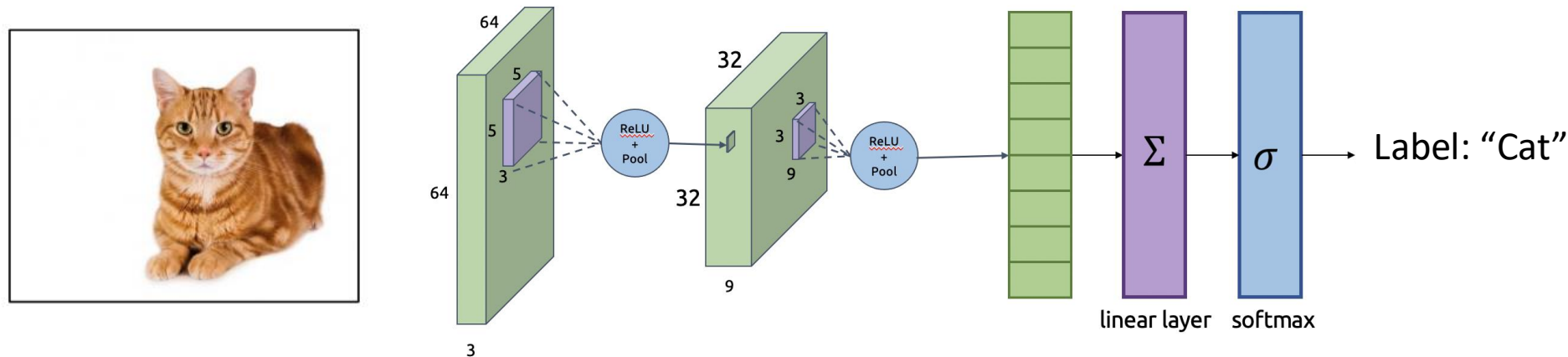


Convolution is
***translation
equivariant***

Max pooling gives
invariance to
small translations

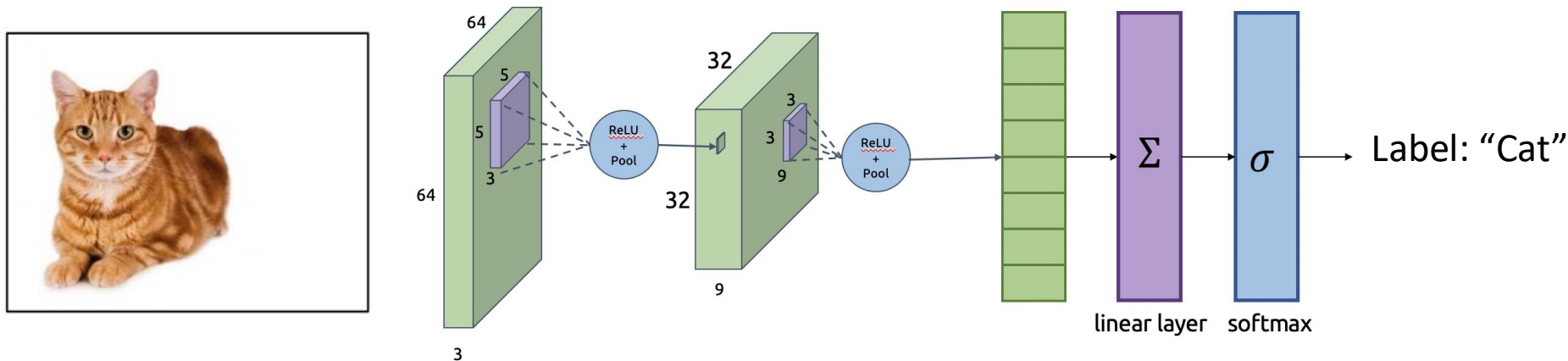
Are CNNs Translation Invariant?

- Answer: CNNs are **“sort of” translation invariant**
 - Shifting the content of the image around tends not to drastically effect the output classification probabilities...



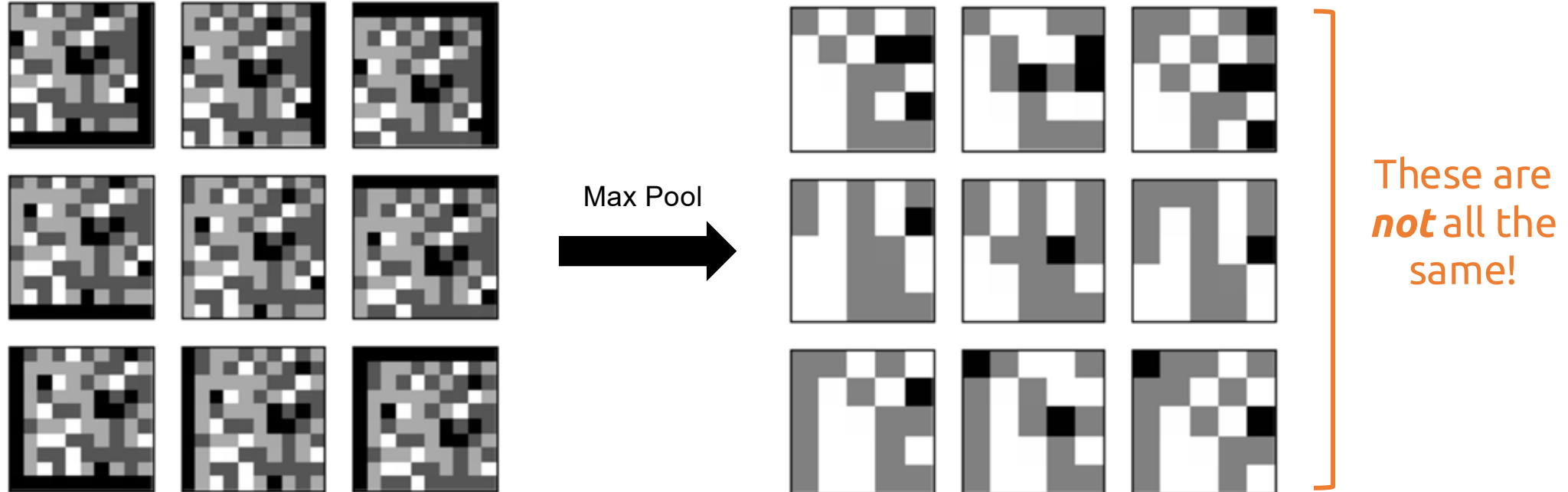
Are CNNs Translation Invariant?

- Answer: CNNs are **“sort of” translation invariant**
 - Shifting the content of the image around tends not to drastically effect the output classification probabilities...



Are CNNs Translation Invariant?

- Answer: CNNs are “sort of” translation invariant
 - Shifting the content of the image around tends not to drastically effect the output classification probabilities...
 - ...but they are **not**, strictly speaking, translation invariant



Are CNNs Translation Invariant?

- Is it possible to build a truly translation invariant CNN?
 - Yes!
 - Have to properly “pre-filter” images before pooling them
 - Comes from signal processing theory (The Sampling Theorem)
 - Take CS 1230 (Computer Graphics) if you want to learn about this!
- One effort to make a translation-invariant CNN:
<https://arxiv.org/pdf/1904.11486.pdf>

Other Invariances

Rotation/Viewpoint Invariance



Other Invariances

Rotation/Viewpoint Invariance



Size Invariance



Other Invariances

Rotation/Viewpoint Invariance



Size Invariance



Illumination Invariance



- All of these are desirable
- How do CNNs fare?
 - Max pooling gives some small amount of size invariance...
 - ...but in general, CNNs don't do well with big changes in size, pose, or lighting
- Consequence of not having these invariances?
 - Need **lots** of training data
 - Have to show the network examples of everything under different poses, lighting, etc.
 - Data Augmentation